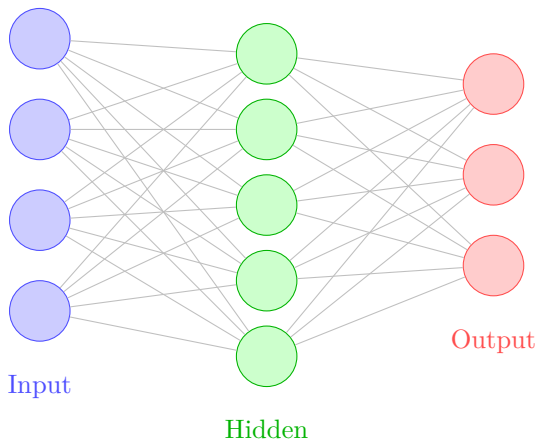
An EduCasheer Initiative

Machine Learning & Deep Learning

Comprehensive Lecture Notes



From Fundamentals to Implementation

Instructor

Musavir Khaliq

Covering:

Machine Learning Fundamentals • Neural Networks • Perceptrons
Gradient Descent • Loss Functions • Backpropagation
Information Theory • Automatic Differentiation

Video Lectures: https://youtube.com/playlist?list=PLG2XU_l-TcR8yWh4nKmPYMX5k9r-1XyBM

January 21, 2026

Contents

1 Introduction to Machine Learning **3**
1.1 Chapter Overview . . . 3
1.2 The Dream of Intelligent Machines . . . 3
1.3 Traditional Programming vs Machine Learning . . . 3
1.3.1 The Traditional Programming Paradigm . . . 3
1.3.2 The Machine Learning Paradigm . . . 4
1.3.3 When Traditional Programming Fails . . . 4
1.4 From Features to Representations . . . 5
1.4.1 Hand-Crafted Features (Classical ML) . . . 5
1.4.2 Representation Learning . . . 5
1.5 Deep Learning: Hierarchical Representations . . . 6
1.5.1 The Limitation of Single-Level Representations . . . 6
1.5.2 The Deep Learning Approach . . . 6
1.5.3 Hierarchical Composition: The Image Example . . . 6
1.6 The AI Landscape: AI, ML, and Deep Learning . . . 7
1.6.1 The Relationship Between Terms . . . 7
1.6.2 A Summary Diagram . . . 7
1.7 The Formal Definition of Learning . . . 8
1.7.1 Tom Mitchell's Definition . . . 8
1.7.2 Example: Learning to Play Chess . . . 8
1.7.3 Example: Email Spam Classification . . . 8
1.8 Types of Machine Learning Tasks . . . 8
1.8.1 Classification . . . 8
1.8.2 Regression . . . 9
1.8.3 Other Important Tasks . . . 9
1.9 Performance Measures . . . 9
1.9.1 Common Metrics . . . 10
1.10 Learning Paradigms: The Experience . . . 10
1.10.1 Supervised Learning . . . 10
1.10.2 Unsupervised Learning . . . 10
1.10.3 Semi-Supervised Learning . . . 11
1.10.4 Reinforcement Learning . . . 11
1.11 The Machine Learning Pipeline . . . 11
1.11.1 Step 1: Problem Definition . . . 11
1.11.2 Step 2: Data Collection . . . 12
1.11.3 Step 3: Data Preprocessing . . . 12
1.11.4 Step 4: Model Selection/Design . . . 12
1.11.5 Step 5: Model Training . . . 12
1.11.6 Step 6: Model Evaluation . . . 12
1.11.7 Step 7: Model Testing . . . 12
1.11.8 Step 8: Deployment and Monitoring . . . 12
1.12 Why Machine Learning is Thriving Now . . . 13
1.12.1 The Three Pillars . . . 13
1.13 Chapter Summary . . . 14

2 Supervised Learning and Curve Fitting **15**

2.1 Chapter Overview . . . 15
2.2 Understanding Data: Patterns and Noise . . . 15
2.2.1 Data as Information . . . 15
2.2.2 The Generating Process . . . 15
2.2.3 Pattern vs. Noise . . . 16
2.3 Synthetic Data Generation . . . 16
2.3.1 The Setup . . . 16
2.3.2 Python Code for Data Generation . . . 16
2.4 The Supervised Learning Problem . . . 17
2.4.1 Problem Formulation . . . 17
2.4.2 Generalization: The Key Challenge . . . 18
2.5 Curve Fitting: A Polynomial Approach . . . 18
2.5.1 Polynomial Basis Functions . . . 18
2.5.2 Choosing the Polynomial Degree . . . 18
2.5.3 The "Curve Fitting Machine" Analogy . . . 19
2.6 Error Functions: Quantifying Fit Quality . . . 19
2.6.1 The Need for an Error Metric . . . 19
2.6.2 Squared Error . . . 19
2.6.3 Geometric Interpretation . . . 20
2.6.4 Mean Squared Error (MSE) and RMSE . . . 20
2.7 Finding the Optimal Parameters . . . 20
2.7.1 The Optimization Problem . . . 20
2.7.2 Closed-Form Solution for Polynomial Fitting . . . 20
2.8 Overfitting and Underfitting . . . 21
2.8.1 The Bias-Variance Tradeoff . . . 21
2.8.2 Visual Comparison . . . 21
2.8.3 The Overfitting Paradox . . . 21
2.8.4 Coefficient Explosion: The Root Cause . . . 21
2.9 Regularization: Controlling Complexity . . . 22
2.9.1 The Key Insight . . . 22
2.9.2 Regularized Error Function . . . 22
2.9.3 The Tug-of-War . . . 22
2.9.4 Effect of Regularization . . . 22
2.10 Hyperparameters and Validation . . . 23
2.10.1 Parameters vs. Hyperparameters . . . 23
2.10.2 Why Not Optimize Hyperparameters Jointly? . . . 23
2.10.3 Train, Validation, and Test Sets . . . 23
2.10.4 The Validation Process . . . 23
2.10.5 Cross-Validation . . . 24
2.11 Chapter Summary . . . 24

3 From Biological to Artificial Neurons 26
3.1 Chapter Overview . . . 26
3.2 The Biological Neuron . . . 26
3.2.1 The Fundamental Unit of Neural Computation . . . 26
3.2.2 Structure of a Biological Neuron . . . 26
3.3 The Artificial Neuron: Computational Analog . . . 27
3.3.1 From Biology to Computation . . . 27
3.3.2 General Structure of an Artificial Neuron . . . 27
3.4 The McCulloch-Pitts Neuron (1943) . . . 28
3.4.1 The First Computational Neuron Model . . . 28
3.4.2 Thresholding Logic . . . 28

3.4.3 Inhibitory and Excitatory Inputs . . . 28
3.5 Implementing Logic Gates . . . 29
3.5.1 The Computational Foundation . . . 29
3.5.2 OR Gate . . . 29
3.5.3 AND Gate . . . 29
3.5.4 NOT Gate . . . 29
3.6 Geometric Interpretation: Linear Separability . . . 29
3.6.1 Decision Boundaries . . . 29
3.6.2 The XOR Problem . . . 30
3.7 The Perceptron (1958) . . . 30
3.7.1 Addressing MP Neuron Limitations . . . 30
3.7.2 Understanding the Bias Term . . . 31
3.7.3 Parameters: What the Perceptron Learns . . . 31
3.8 The Perceptron Learning Algorithm . . . 31
3.8.1 Learning from Data . . . 31
3.8.2 The Algorithm . . . 31
3.8.3 Convergence Theorem . . . 32
3.9 Chapter Summary . . . 32

4 Activation Functions and Sigmoid Neurons 34
4.1 Chapter Overview . . . 34
4.2 From Step Functions to Smooth Activations . . . 34
4.2.1 Problems with the Step Function . . . 34
4.2.2 Desirable Properties of Activation Functions . . . 35
4.3 The Sigmoid Function . . . 35
4.3.1 Definition . . . 35
4.3.2 Properties of Sigmoid . . . 35
4.3.3 Derivative of Sigmoid . . . 35
4.3.4 Python Implementation . . . 36
4.4 The Sigmoid Neuron . . . 36
4.4.1 Architecture . . . 36
4.4.2 Interpreting Sigmoid Output . . . 37
4.5 Loss Functions . . . 37
4.5.1 Why Loss Functions? . . . 37
4.5.2 Mean Squared Error (MSE) . . . 37
4.5.3 Properties of MSE . . . 37
4.6 The Forward Pass . . . 38
4.6.1 Definition . . . 38
4.6.2 Example: Forward Pass by Hand . . . 38
4.7 Other Activation Functions . . . 38
4.7.1 Tanh (Hyperbolic Tangent) . . . 38
4.7.2 ReLU (Rectified Linear Unit) . . . 38
4.7.3 Comparison . . . 39
4.8 Chapter Summary . . . 39

5 Multi-Layer Perceptrons 40
5.1 Chapter Overview . . . 40
5.2 The Need for Multiple Layers . . . 40
5.2.1 The XOR Revisited . . . 40
5.3 Neural Networks as Graphs . . . 41
5.3.1 Directed Acyclic Graphs (DAGs) . . . 41
5.3.2 Defining Depth . . . 41

5.3.3 What Makes a Network "Deep"? . . . 41
5.3.4 Defining Layers . . . 42
5.4 Standard Neural Network Notation . . . 42
5.4.1 Network Structure . . . 42
5.4.2 Weight Matrix Notation . . . 42
5.4.3 Bias Vector Notation . . . 43
5.4.4 Activation Notation . . . 43
5.4.5 Summary of Notation . . . 43
5.5 Forward Propagation in MLPs . . . 43
5.5.1 The Algorithm . . . 43
5.5.2 Matrix Formulation . . . 44
5.5.3 Python Implementation . . . 44
5.6 Representational Power . . . 44
5.6.1 Universal Approximation Theorem . . . 44
5.6.2 Why Depth Helps . . . 45
5.7 Chapter Summary . . . 45

6 Gradient Descent and Optimization 47
6.1 Chapter Overview . . . 47
6.2 Formalizing the Learning Problem . . . 47
6.2.1 The True Goal . . . 47
6.2.2 Two Fundamental Problems . . . 47
6.2.3 The Solution: Empirical Risk . . . 47
6.3 Divergence Functions and Losses . . . 48
6.3.1 Properties of a Good Divergence . . . 48
6.3.2 Common Loss Functions . . . 48
6.4 Gradient Descent . . . 48
6.4.1 The Optimization Problem . . . 48
6.4.2 The Key Insight: Gradients Point Uphill . . . 49
6.4.3 The Update Rule . . . 49
6.4.4 The Gradient Descent Algorithm . . . 49
6.5 The Learning Rate . . . 49
6.5.1 Choosing the Learning Rate . . . 49
6.6 Variants of Gradient Descent . . . 50
6.6.1 Batch Gradient Descent . . . 50
6.6.2 Stochastic Gradient Descent (SGD) . . . 50
6.6.3 Mini-Batch Gradient Descent . . . 50
6.7 Computing Gradients . . . 50
6.7.1 The Challenge . . . 50
6.7.2 The Solution: Backpropagation . . . 50
6.8 Implementation . . . 51
6.9 Chapter Summary . . . 52

7 Backpropagation 53
7.1 Motivation and Overview . . . 53
7.1.1 Independent vs. Dependent Parameters . . . 53
7.2 Review of Calculus Fundamentals . . . 53
7.2.1 The Derivative as Rate of Change . . . 53
7.2.2 Partial Derivatives and Multivariate Functions . . . 54
7.2.3 The Chain Rule . . . 54
7.2.4 Chain Rule with Multiple Paths . . . 54
7.3 Computational Graphs . . . 55

7.4 Forward Pass: Notation and Review . . . 55
7.4.1 Notation Summary . . . 55
7.4.2 Forward Pass Equations . . . 56
7.5 The Error (Delta) Vector . . . 56
7.6 Backpropagation Derivation . . . 56
7.6.1 Step 1: Error at the Output Layer . . . 56
7.6.2 Step 2: Backpropagating the Error . . . 57
7.6.3 Step 3: Gradients with Respect to Parameters . . . 58
7.7 Summary: The Backpropagation Algorithm . . . 59
7.8 Worked Example: Backpropagation by Hand . . . 59
7.8.1 Network Setup . . . 59
7.8.2 Forward Pass . . . 59
7.8.3 Backward Pass . . . 60
7.8.4 Parameter Gradients . . . 61
7.8.5 Gradient Descent Update . . . 61
7.9 Mini-batch Gradient Descent . . . 61
7.10 Why is it Called "Backpropagation"? . . . 61
7.11 Sigmoid Derivative . . . 62
7.12 Key Takeaways . . . 62

8 Information Theory and Loss Functions 64
8.1 The Learning Problem . . . 64
8.1.1 Setup . . . 64
8.1.2 The Divergence Problem . . . 64
8.2 Information and Surprise . . . 64
8.2.1 Intuition: Data as Surprise . . . 64
8.2.2 Defining Information . . . 65
8.3 Entropy . . . 65
8.3.1 Definition . . . 65
8.3.2 Properties of Entropy . . . 65
8.3.3 Examples . . . 66
8.3.4 Entropy Visualization . . . 66
8.4 Cross-Entropy . . . 66
8.4.1 Motivation . . . 66
8.4.2 Key Insight . . . 67
8.5 KL Divergence . . . 67
8.5.1 Definition . . . 67
8.5.2 Properties . . . 67
8.5.3 Why Cross-Entropy for Loss? . . . 67
8.6 The Softmax Function . . . 68
8.6.1 Properties . . . 68
8.6.2 Example . . . 68
8.6.3 Softmax Temperature . . . 68
8.7 Cross-Entropy Loss for Classification . . . 68
8.7.1 Binary Classification: Binary Cross-Entropy (BCE) . . . 68
8.7.2 Multi-class Classification: Categorical Cross-Entropy (CCE) . . . 69
8.7.3 Example: Multi-class Classification . . . 69
8.8 Maximum Likelihood Estimation Perspective . . . 69
8.8.1 The Four-Step Recipe for Loss Construction . . . 69
8.8.2 Deriving Binary Cross-Entropy . . . 70
8.8.3 Deriving Categorical Cross-Entropy . . . 70
8.8.4 MSE from Gaussian Assumption . . . 70

8.9 Gradient of Cross-Entropy with Softmax . . . 71
8.10 Why Use Logarithms? . . . 71
8.11 Summary: Loss Function Selection . . . 71
8.12 Practical Considerations . . . 72
8.12.1 Label Smoothing . . . 72
8.12.2 Numerical Stability . . . 72

9 Practical Deep Learning 73
9.1 From Theory to Implementation . . . 73
9.1.1 Three Types of Differentiation . . . 73
9.1.2 Numerical Differentiation Details . . . 73
9.2 Automatic Differentiation (Autograd) . . . 74
9.2.1 The Core Idea . . . 74
9.2.2 Forward Mode vs. Reverse Mode . . . 74
9.3 Building a Simple Autograd System . . . 74
9.3.1 The Value Class . . . 75
9.3.2 Using Our Autograd Engine . . . 76
9.4 Building a Neural Network from Scratch . . . 77
9.4.1 Neuron Class . . . 77
9.4.2 Layer Class . . . 77
9.4.3 MLP Class . . . 77
9.5 The Training Loop . . . 78
9.6 The Complete Training Algorithm . . . 79
9.7 Hyperparameters . . . 80
9.7.1 Key Hyperparameters . . . 80
9.7.2 Learning Rate . . . 80
9.8 Common Challenges . . . 80
9.8.1 Vanishing Gradients . . . 80
9.8.2 Exploding Gradients . . . 81
9.8.3 Overfitting . . . 81
9.8.4 Underfitting . . . 81
9.9 Weight Initialization . . . 81
9.9.1 Xavier/Glorot Initialization . . . 81
9.9.2 He Initialization . . . 82
9.10 Regularization Techniques . . . 82
9.10.1 L2 Regularization (Weight Decay) . . . 82
9.10.2 Dropout . . . 82
9.11 Tips for Success . . . 82
9.12 From Scratch to Frameworks . . . 83
9.13 Key Takeaways . . . 84

Chapter 1

Introduction to Machine Learning

1.1 Chapter Overview

Machine learning represents a paradigm shift in how we approach problem-solving with computers. This chapter introduces the fundamental concepts of machine learning, contrasts it with traditional programming approaches, and establishes the vocabulary and mental frameworks necessary for understanding the deeper concepts that follow.

Learning Objectives:

- Understand the motivation behind machine learning
- Distinguish between traditional programming and machine learning paradigms
- Define representation learning and deep learning
- Learn the formal definition of machine learning (Tom Mitchell's definition)
- Identify different types of ML tasks, performance measures, and learning paradigms
- Understand the typical ML/DL pipeline

1.2 The Dream of Intelligent Machines

The aspiration to create machines that can think, reason, and learn like humans has driven researchers for decades. The fundamental question is: *Can we somehow formalize thinking itself?* Can we create mathematical or algorithmic representations of the cognitive processes that humans perform naturally?

Historical Context: The Quest for Artificial Intelligence

The dream of creating intelligent machines dates back centuries, but the formal field of Artificial Intelligence was established at the Dartmouth Conference in 1956. Early pioneers like Alan Turing, John McCarthy, and Marvin Minsky laid the groundwork for what would become one of the most transformative technologies of our time.
Turing proposed the famous "Turing Test" in 1950, asking whether a machine could exhibit intelligent behavior indistinguishable from a human. This question still resonates today as we witness large language models engaging in remarkably human-like conversations.

1.3 Traditional Programming vs Machine Learning

1.3.1 The Traditional Programming Paradigm

In traditional programming, a human programmer writes explicit rules that the computer follows. The flow is:

Traditional Programming Example

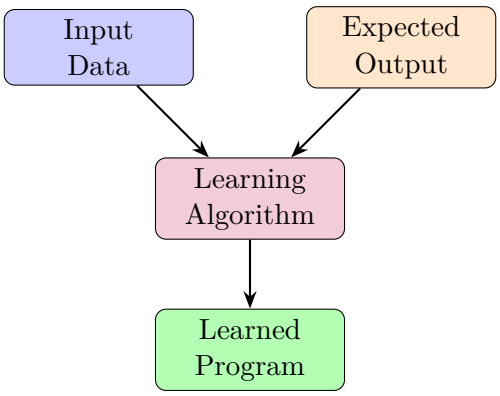
Consider sorting numbers. A programmer explicitly writes rules like:

1. Compare adjacent elements
2. If out of order, swap them
3. Repeat until no swaps needed

Every possible scenario must be anticipated and coded.

1.3.2 The Machine Learning Paradigm

Machine learning inverts this approach. Instead of programming explicit rules, we provide *examples* and let the algorithm discover patterns:



Machine Learning

Machine learning is the field of study that gives computers the ability to learn patterns from data without being explicitly programmed for each specific task. The computer discovers the "program" (the mapping from inputs to outputs) by analyzing examples.

1.3.3 When Traditional Programming Fails

Traditional rule-based programming works well for well-defined problems with clear logic. However, many real-world tasks are extremely difficult or impossible to define with explicit rules:

- **Image Recognition**: How do you write rules to identify a cat in an image? Consider all possible poses, lighting conditions, occlusions, and cat breeds.
- **Speech Recognition**: The same word can sound vastly different based on accent, speed, pitch, and background noise.
- **Natural Language Understanding**: Language is ambiguous, context-dependent, and constantly evolving.

The Curse of Enumeration

Consider the task of identifying a car in an image. One might attempt to define rules:

- "Look for four wheels"
- "Check for a windshield"

- "Identify the body shape"

But what about:

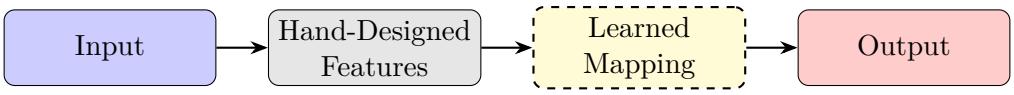
- A car covered in snow (no visible wheels)?
- A car reflected in water?
- A sports car vs. an SUV vs. a vintage automobile?
- A car at an unusual angle?

The number of possible variations is virtually infinite. This is why **learning from examples** is more powerful than **enumerating rules**.

1.4 From Features to Representations

1.4.1 Hand-Crafted Features (Classical ML)

In classical machine learning, domain experts design *features*—measurable properties of the data that are relevant to the task. The machine learning algorithm then learns a mapping from these features to the output.

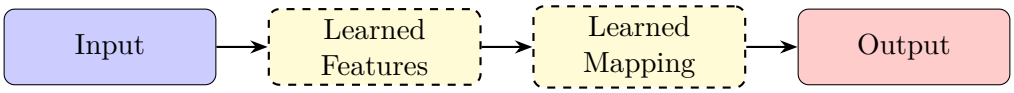


Example: For classifying flowers (the Iris dataset):

- Feature 1: Sepal length
- Feature 2: Sepal width
- Feature 3: Petal length
- Feature 4: Petal width

1.4.2 Representation Learning

Representation learning goes a step further: instead of relying on hand-crafted features, the algorithm learns the features (representations) themselves.



The Power of Learned Representations

Why is representation learning important?

1. **Eliminates Domain Expertise Bottleneck**: Feature engineering requires deep domain knowledge. Representation learning can discover relevant features automatically.
2. **Discovers Non-Obvious Patterns**: Humans might miss important features that the algorithm can discover.
3. **Transfers Across Tasks**: Good representations learned for one task often help with related tasks.

1.5 Deep Learning: Hierarchical Representations

1.5.1 The Limitation of Single-Level Representations

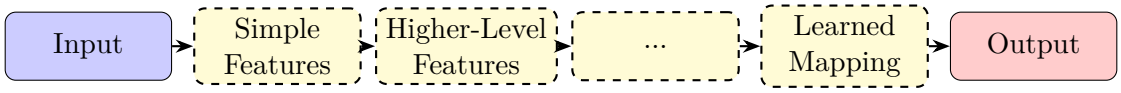
Consider the car recognition problem. The same car can appear very different under various conditions:

- Different viewing angles
- Various lighting conditions
- Partial occlusion
- Different backgrounds

A single layer of learned features struggles to capture all these variations.

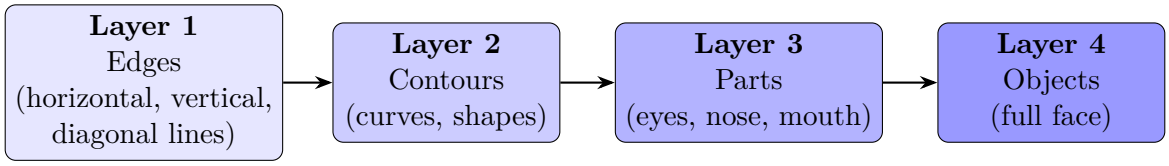
1.5.2 The Deep Learning Approach

Deep learning extends representation learning by learning *multiple layers* of increasingly abstract representations:



1.5.3 Hierarchical Composition: The Image Example

Consider how a deep learning system might process an image for face recognition:



Deep Learning

Deep learning is a subset of machine learning that uses neural networks with multiple layers (depth > 2) to learn hierarchical representations of data. Each layer transforms the representation into something more abstract, eventually producing features suitable for the final task.

Why "Deep"? The term refers to the depth of the network—the number of layers through which the data passes. Deeper networks can learn more complex, abstract representations.

The Book Analogy

Consider how a book is constructed:

$$\text{Letters} \rightarrow \text{Words} \rightarrow \text{Sentences} \rightarrow \text{Paragraphs} \rightarrow \text{Chapters} \rightarrow \text{Book}$$

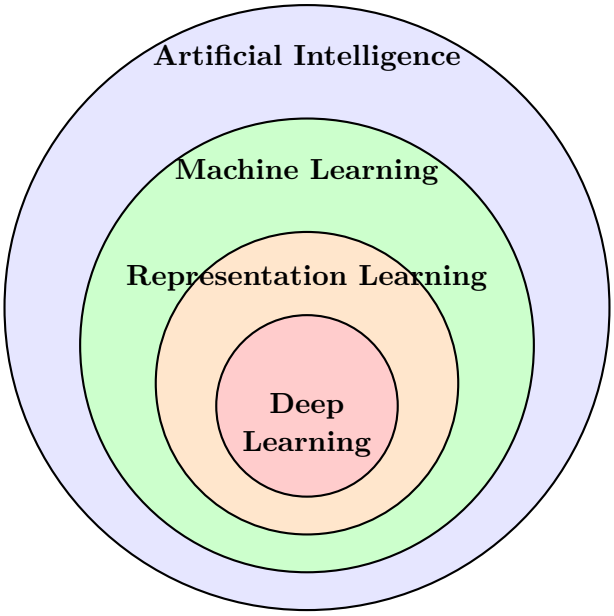
Deep learning operates similarly. At each layer:

- Simple elements (letters/edges) combine to form
- More complex structures (words/contours), which combine to form
- Higher-level concepts (sentences/object parts), ultimately forming
- Complete understanding (chapters/full objects)

This **hierarchical composition** is key to why deep learning works so well.

1.6 The AI Landscape: AI, ML, and Deep Learning

1.6.1 The Relationship Between Terms



- **Artificial Intelligence (AI)**: The broadest category—any system that exhibits intelligent behavior. This includes rule-based systems, expert systems, and learning-based approaches.
- **Machine Learning (ML)**: A subset of AI where systems learn patterns from data rather than being explicitly programmed.
- **Representation Learning**: A subset of ML where the system learns the features/representations of data automatically.
- **Deep Learning (DL)**: A subset of representation learning using multi-layered neural networks to learn hierarchical representations.

1.6.2 A Summary Diagram

The evolution from rule-based to deep learning systems:

Paradigm	Input	Features	Mapping	Output
Rule-based	Given	Hand-designed	Hand-designed	✓
Classical ML	Given	Hand-designed	Learned	✓
Rep. Learning	Given	Learned	Learned	✓
Deep Learning	Given	Learned (Hierarchical)	Learned	✓

Historical Note: The AI Winters

The history of AI has seen periods of great hope followed by disappointment:
1950s-1960s: Early enthusiasm. Researchers promised thinking machines were imminent.
1970s (First AI Winter): The Perceptron was shown unable to solve XOR. Funding

dried up.
1980s: Expert systems revival, multi-layer networks developed.
Late 1980s-1990s (Second AI Winter): Expert systems proved brittle and hard to maintain.
2012-Present: Deep learning renaissance. AlexNet won ImageNet, GPT and large language models emerged.
The field underwent multiple name changes: Cybernetics → Connectionism → Neural Networks → Machine Learning → Deep Learning, partly to escape the stigma of previous "AI winters."

1.7 The Formal Definition of Learning

1.7.1 Tom Mitchell's Definition

The computer scientist Tom Mitchell provided one of the most widely cited definitions of machine learning:

Tom Mitchell's Definition of Learning (1997)

A computer program is said to **learn** from experience **E** with respect to some class of tasks **T** and performance measure **P**, if its performance at tasks in **T**, as measured by **P**, improves with experience **E**.

This definition identifies three essential components of any learning system:

1. **Task (T)**: What the system is trying to accomplish
2. **Performance Measure (P)**: How we quantify success
3. **Experience (E)**: The data the system learns from

1.7.2 Example: Learning to Play Chess

- **Task (T)**: Play chess (make legal moves, try to win)
- **Performance (P)**: Win rate against opponents
- **Experience (E)**: Playing games, analyzing past games

If, after playing many games, the win rate improves, then the program has *learned*.

1.7.3 Example: Email Spam Classification

- **Task (T)**: Classify emails as spam or not-spam
- **Performance (P)**: Accuracy (percentage correctly classified)
- **Experience (E)**: A dataset of labeled emails

1.8 Types of Machine Learning Tasks

1.8.1 Classification

The task of assigning an input to one of k discrete categories.

Classification

A function $f : \mathbb{R}^n \to \{1, 2, \ldots, k\}$ that maps an n-dimensional feature vector to one of k class labels.

Examples:

- Image classification (cat vs. dog)
- Spam detection (spam vs. not-spam)
- Medical diagnosis (disease A, B, C, or healthy)
- Handwritten digit recognition (0-9)

1.8.2 Regression

The task of predicting a continuous numerical value.

Regression

A function $f : \mathbb{R}^n \to \mathbb{R}$ that maps an n-dimensional feature vector to a real-valued output.

Examples:

- Stock price prediction
- House price estimation
- Temperature forecasting
- Age estimation from photographs

1.8.3 Other Important Tasks

- **Classification with Missing Values**: Classification when some features are unavailable
- **Transcription**: Converting unstructured data to structured form (e.g., speech-to-text, image captioning)
- **Machine Translation**: Converting text from one language to another
- **Structured Output**: Outputs with relationships between elements (e.g., parsing sentences)
- **Anomaly Detection**: Identifying unusual patterns (e.g., fraud detection)
- **Synthesis/Sampling**: Generating new data similar to training data (e.g., image generation)
- **Imputation**: Filling in missing values
- **Denoising**: Removing noise from signals (e.g., image deblurring)
- **Density Estimation**: Learning the probability distribution of data

1.9 Performance Measures

Performance measures (metrics) quantify how well a model performs its task.

1.9.1 Common Metrics

- **Accuracy**:
$$\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Predictions}}$$
- **Error Rate**: $1 - \text{Accuracy}$
- **Precision**: Of all positive predictions, how many were correct?
$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$
- **Recall**: Of all actual positives, how many were detected?
$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$
- **F1 Score**: Harmonic mean of precision and recall
$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$
- **AUC-ROC**: Area Under the Receiver Operating Characteristic Curve

Choosing the Right Metric

The choice of metric depends on the problem:
- For balanced datasets: Accuracy often suffices
- For imbalanced data: Precision, Recall, F1 are more informative
- For ranking problems: AUC-ROC is preferred
- For regression: Mean Squared Error (MSE), Mean Absolute Error (MAE)

In fraud detection, for example, accuracy might be misleading—99% accuracy could mean missing all fraud cases if fraud is only 1% of transactions!

1.10 Learning Paradigms: The Experience

How does a machine learning system gain "experience"? This depends on the type of data and feedback available.

1.10.1 Supervised Learning

The algorithm learns from labeled examples.

Supervised Learning

Given a dataset $\mathcal{D} = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \ldots, (\mathbf{x}_n, y_n)\}$ where each $\mathbf{x}_i$ is a feature vector and y_i is the corresponding label, learn a function f that maps inputs to outputs.

Examples: Classification, regression

1.10.2 Unsupervised Learning

The algorithm learns patterns without labeled outputs.

Unsupervised Learning

Given a dataset $\mathcal{D} = \{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n\}$ (no labels), discover the underlying structure or patterns in the data.

Examples: Clustering, dimensionality reduction, density estimation

1.10.3 Semi-Supervised Learning

Some examples are labeled, others are not. This is common in practice (labeling is expensive).

1.10.4 Reinforcement Learning

An agent learns by interacting with an environment and receiving rewards.

Reinforcement Learning

An **agent** takes **actions** in an **environment**, transitioning between **states**. Each transition yields a **reward**. The agent learns a **policy** that maximizes cumulative reward.

Key concepts:

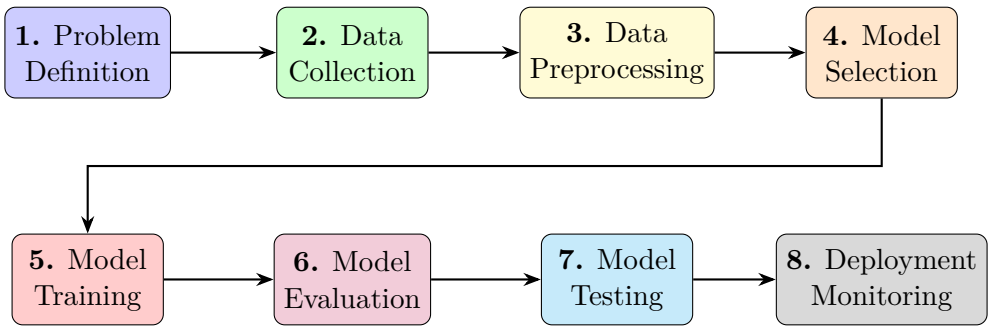
- **Exploration**: Trying new actions to discover their effects
- **Exploitation**: Using known good actions to maximize reward
- **Exploration-Exploitation Trade-off**: Balancing discovery with optimization

Examples:

- AlphaGo: Learning to play the game Go
- Robotic control: Learning to walk
- RLHF (Reinforcement Learning from Human Feedback): Training ChatGPT

1.11 The Machine Learning Pipeline

A typical ML/DL project follows a structured pipeline:



1.11.1 Step 1: Problem Definition

- Define the task (classification, regression, etc.)
- Clarify the objective (what does success look like?)
- Understand the business or scientific context

1.11.2 Step 2: Data Collection

- Identify data sources (databases, APIs, sensors, web scraping)
- Gather relevant data
- Format data appropriately (CSV, JSON, images, etc.)

1.11.3 Step 3: Data Preprocessing

- **Data Cleaning**: Handle missing values, remove duplicates, fix inconsistencies
- **Feature Engineering**: Create new features, encode categorical variables, scale/normalize
- **Data Splitting**: Divide into training, validation, and test sets
- **Data Augmentation**: Artificially expand the dataset (e.g., flip images, add noise)

Garbage In, Garbage Out

Data preprocessing is often where 60% or more of project time is spent. The quality of your data directly determines the quality of your model. No amount of sophisticated algorithms can compensate for poor data.

1.11.4 Step 4: Model Selection/Design

- Choose appropriate algorithms (Decision Trees, Neural Networks, SVMs, etc.)
- Design architecture (for deep learning: number of layers, neurons, etc.)

1.11.5 Step 5: Model Training

- Choose a loss function (how to measure error)
- Select an optimizer (SGD, Adam, etc.)
- Fit the model to training data
- Use batching for large datasets

1.11.6 Step 6: Model Evaluation

- Compute performance metrics on validation set
- Perform cross-validation
- Hyperparameter tuning (learning rate, regularization, architecture choices)

1.11.7 Step 7: Model Testing

- Evaluate on held-out test set
- Check for overfitting (memorizing training data) or underfitting (poor performance)

1.11.8 Step 8: Deployment and Monitoring

- Export the model
- Deploy via APIs (Flask, FastAPI)

- Monitor performance in production
- Retrain as needed (handle model drift)

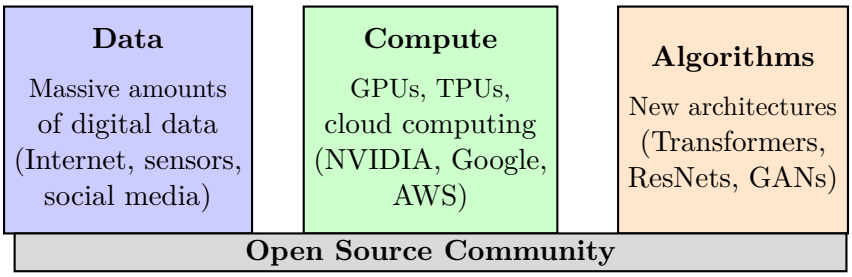
Fraud Detection Pipeline

Problem: Detect fraudulent credit card transactions
Data Collection: Obtain transaction records from banks (customer ID, amount, time, merchant, etc.)
Preprocessing: Handle missing timestamps, create features (transaction frequency, deviation from average amount), encode merchant categories
Model: XGBoost or deep neural network
Training: Binary cross-entropy loss, Adam optimizer
Evaluation: Precision, Recall, F1 Score (accuracy alone is misleading due to class imbalance)
Deployment: Real-time API that scores each transaction
Monitoring: Track model drift as fraudsters adapt their strategies

1.12 Why Machine Learning is Thriving Now

Many core algorithms (including backpropagation) were developed in the 1970s-1980s. So why has ML/DL taken off only recently?

1.12.1 The Three Pillars



1. **Data**: The internet age has produced unprecedented amounts of data. Web scraping, social media, IoT sensors—data is everywhere.
2. **Compute Power**: GPUs (Graphics Processing Units) enable massive parallelization. What took weeks now takes hours. NVIDIA's GPUs have become the "shovels in the AI gold rush."
3. **Algorithmic Advances**: New architectures like Transformers, ResNets, and techniques like dropout and batch normalization have overcome previous limitations.
4. **Open Source**: Frameworks like TensorFlow, PyTorch, and pre-trained models allow rapid experimentation. The community shares knowledge freely.

The Shovel Sellers in a Gold Rush

There's a famous business principle: "In a gold rush, don't mine for gold—sell shovels." NVIDIA exemplified this perfectly. While companies race to build the best AI models, NVIDIA provides the hardware (GPUs) that everyone needs. This is why NVIDIA's valuation skyrocketed during the AI boom.

1.13 Chapter Summary

- **Machine Learning** is a paradigm where computers learn patterns from data rather than being explicitly programmed.
- **Representation Learning** allows algorithms to discover the features themselves, rather than relying on hand-crafted features.
- **Deep Learning** uses multiple layers to learn hierarchical representations, enabling handling of complex, high-dimensional data.
- Tom Mitchell's definition formalizes learning: improvement on task T with respect to measure P through experience E.
- ML tasks include classification, regression, transcription, translation, anomaly detection, and more.
- Learning paradigms: supervised (labeled data), unsupervised (no labels), semi-supervised (partial labels), reinforcement (rewards).
- The ML pipeline: Problem Definition $\rightarrow$ Data Collection $\rightarrow$ Preprocessing $\rightarrow$ Model Selection $\rightarrow$ Training $\rightarrow$ Evaluation $\rightarrow$ Testing $\rightarrow$ Deployment.
- ML/DL is thriving due to abundant data, powerful compute, algorithmic innovations, and open-source culture.

Chapter 2

Supervised Learning and Curve Fitting

2.1 Chapter Overview

This chapter introduces supervised learning through the lens of **curve fitting**—one of the simplest yet most illustrative examples in machine learning. We will explore how data is generated, what patterns and noise mean, how to fit mathematical functions to data, and the fundamental tension between fitting data well (minimizing error) and generalizing to new, unseen examples.

Learning Objectives:

- Understand how real-world data combines patterns and noise
- Formulate curve fitting as an optimization problem
- Define and compute error functions (squared error, MSE, RMSE)
- Distinguish between underfitting, good fit, and overfitting
- Introduce regularization as a solution to overfitting
- Understand the role of hyperparameters and validation

2.2 Understanding Data: Patterns and Noise

2.2.1 Data as Information

In machine learning, data is the raw material from which we extract knowledge. Each data point carries *information*—the degree to which observing that data point changes what we know about the world.

Information as Surprise

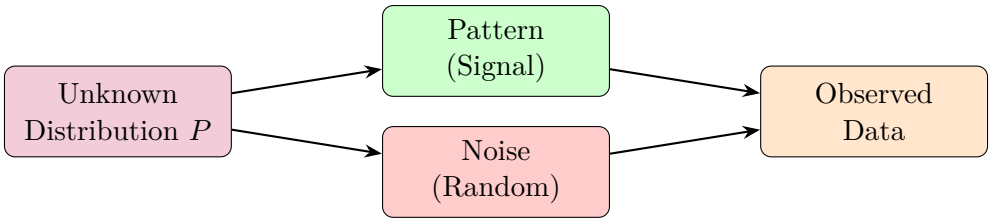
The concept of **information** is deeply connected to **surprise**:

- If you observe something you already expected, you gain little information.
- If you observe something unexpected, you gain more information.

This intuition forms the basis of **information theory**, which we'll explore later. For now, understand that data contains hidden patterns that we wish to discover.

2.2.2 The Generating Process

We assume that data comes from some underlying **generating process** or **distribution**. This distribution is typically *unknown*—if we knew it, there would be no need for machine learning!



2.2.3 Pattern vs. Noise

Real-world data consists of two components:

1. **Pattern (Signal)**: The underlying regularity or structure we want to learn. This is captured by some function $g(\mathbf{x})$.
2. **Noise**: Random variations that obscure the pattern. Noise arises from:
   - Measurement errors
   - Intrinsic randomness in the process
   - Unobserved variables affecting the outcome

Data Generation Model

In a typical supervised learning scenario, targets are generated as:

$$t = g(\mathbf{x}) + \epsilon$$

where:
- $g(\mathbf{x})$ is the true (unknown) generating function
- ϵ is noise, often assumed to be $\epsilon \sim \mathcal{N}(0, \sigma^2)$ (Gaussian with mean 0)

Our Goal: Learn the pattern $g(\mathbf{x})$ while ignoring the noise.

2.3 Synthetic Data Generation

To understand supervised learning concretely, let's work with synthetic data where we know the true generating function.

2.3.1 The Setup

Consider:

- Input variable: $x \in [0, 1]$
- Target variable: $t \in \mathbb{R}$
- Generating function: $g(x) = \sin(2\pi x)$
- Noise: $\epsilon \sim \mathcal{N}(0, 0.1^2)$

Thus, the observed targets are:

$$t_n = \sin(2\pi x_n) + \epsilon_n$$

2.3.2 Python Code for Data Generation

```python
import numpy as np
import matplotlib.pyplot as plt

def generate_synthetic_data(n, noise_std=0.1, random_seed=None):
    """Generate synthetic data from sin(2*pi*x) with Gaussian
        noise."""
    if random_seed is not None:
        np.random.seed(random_seed)

    # Generate uniformly spaced x values in [0, 1]
    x = np.linspace(0, 1, n)

    # Compute the true target (the pattern)
    t_true = np.sin(2 * np.pi * x)

    # Add Gaussian noise
    noise = np.random.normal(0, noise_std, n)
    t = t_true + noise

    return x, t
```

Listing 2.1: Synthetic Data Generation

Why Synthetic Data?

By using synthetic data where we know the true generating function:

1. We can **verify** whether our learned model approximates the true function
2. We can **visualize** the gap between predictions and truth
3. We can **control** the noise level to study its effects

In real applications, we never know $g(\mathbf{x})$—this is precisely what we're trying to learn!

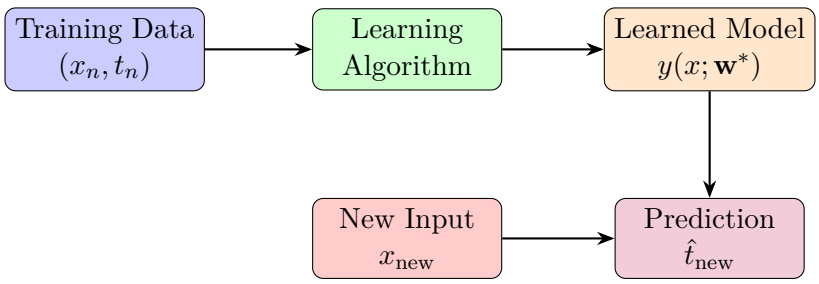
2.4 The Supervised Learning Problem

2.4.1 Problem Formulation

We have:

- A **training set**: $\mathcal{D} = \{(x_1, t_1), (x_2, t_2), \ldots, (x_N, t_N)\}$
- Input features: x_n (known)
- Targets: t_n (known for training data)

Goal: Learn a function $y(x; \mathbf{w})$ that predicts targets for *new, unseen* inputs.



2.4.2 Generalization: The Key Challenge

The core difficulty is that we have **finite data** but want to make predictions for **infinitely many possible inputs**. This requires **generalization**—the model must capture the underlying pattern, not memorize the training examples.

The Generalization Principle

We want to generalize, not memorize.
A model that perfectly fits training data but fails on new data has *memorized* rather than *learned*. The true test of learning is performance on unseen data.

2.5 Curve Fitting: A Polynomial Approach

2.5.1 Polynomial Basis Functions

One of the simplest approaches to fitting a function is using **polynomials**. We express our model as:

Polynomial Model

$$y(x; \mathbf{w}) = w_0 + w_1 x + w_2 x^2 + \cdots + w_M x^M = \sum_{j=0}^{M} w_j x^j$$

where:

- M is the **degree** of the polynomial
- $\mathbf{w} = (w_0, w_1, \ldots, w_M)^\top$ are the **coefficients** (parameters) to learn

Why Polynomials?

- **Universal Approximation**: By Taylor's theorem, any smooth function can be approximated arbitrarily well by a polynomial of sufficient degree.
- **Linear in Parameters**: Although $y(x; \mathbf{w})$ is nonlinear in x, it is *linear in* $\mathbf{w}$. This makes optimization tractable.
- **Simple to Understand**: Polynomials provide intuitive examples for understanding key concepts.

2.5.2 Choosing the Polynomial Degree

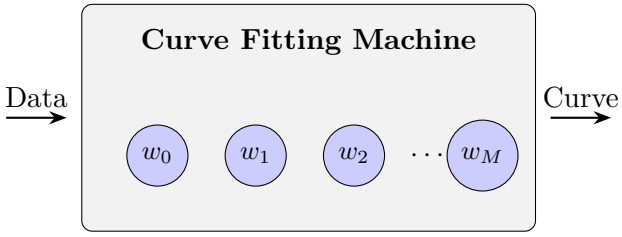
The degree M controls model complexity:

- $M = 0$: Constant (horizontal line)
- $M = 1$: Linear (straight line)
- $M = 2$: Quadratic (parabola)
- $M = 3$: Cubic
- Higher M: More complex curves

As M increases, the polynomial gains more **degrees of freedom** to fit the data.

2.5.3 The "Curve Fitting Machine" Analogy

Imagine a machine with **knobs** labeled $w_0, w_1, \ldots, w_M$. By adjusting these knobs, we change the shape of the curve. Our goal is to find the knob settings that make the curve pass as close as possible to all data points.



2.6 Error Functions: Quantifying Fit Quality

2.6.1 The Need for an Error Metric

Given a set of parameters $\mathbf{w}$, how do we measure how well the resulting curve fits the data? We need a **quantitative measure** of the discrepancy between predictions and true targets.

2.6.2 Squared Error

The most common choice is the **sum of squared errors**:

Sum of Squared Errors (SSE)

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^{N} (y(x_n; \mathbf{w}) - t_n)^2$$

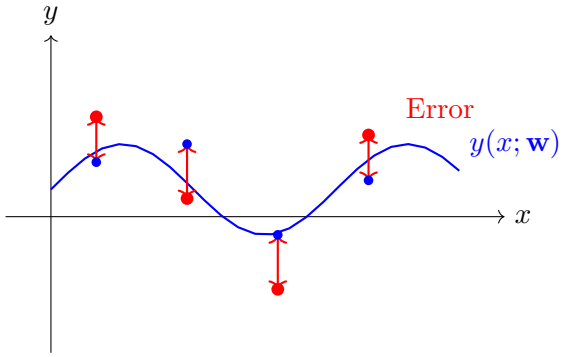
where:

- $y(x_n; \mathbf{w})$ is the prediction for input x_n
- t_n is the true target
- The factor $\frac{1}{2}$ is for mathematical convenience (cancels when differentiating)

Why Squared Error?

1. **Non-negative**: Squared values are always ≥ 0
2. **Zero at perfect fit**: $E(\mathbf{w}) = 0$ when predictions exactly match targets
3. **Penalizes large errors more**: Squaring magnifies large deviations
4. **Differentiable**: Smooth function, amenable to calculus-based optimization

2.6.3 Geometric Interpretation



Each red line represents the error $(y(x_n; \mathbf{w}) - t_n)$. The squared error sums the squares of all these vertical distances.

2.6.4 Mean Squared Error (MSE) and RMSE

Mean Squared Error and Root Mean Squared Error

Mean Squared Error (MSE):

$$\text{MSE} = \frac{1}{N} \sum_{n=1}^{N} (y(x_n; \mathbf{w}) - t_n)^2$$

Root Mean Squared Error (RMSE):

$$E_{\text{RMS}} = \sqrt{\frac{1}{N} \sum_{n=1}^{N} (y(x_n; \mathbf{w}) - t_n)^2}$$

The RMSE has the same units as the target variable, making it more interpretable.

2.7 Finding the Optimal Parameters

2.7.1 The Optimization Problem

We want to find the parameters $\mathbf{w}^*$ that minimize the error function:

$$\mathbf{w}^* = \arg\min_{\mathbf{w}} E(\mathbf{w})$$

2.7.2 Closed-Form Solution for Polynomial Fitting

For polynomial regression with squared error, the solution can be found analytically. The normal equations yield:

$$\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{t}$$

where $\mathbf{X}$ is the **design matrix** with rows containing $(1, x_n, x_n^2, \ldots, x_n^M)$.

NumPy Implementation

In practice, we use optimized implementations:

```python
import numpy as np

# Fit polynomial and get coefficients
coefficients = np.polyfit(x, t, degree)

# Create polynomial function
polynomial = np.poly1d(coefficients)

# Make predictions
predictions = polynomial(x_new)
```

2.8 Overfitting and Underfitting

2.8.1 The Bias-Variance Tradeoff

When we change the polynomial degree M, we observe a fundamental tension:

- **Low M (Underfitting)**: Model is too simple to capture the true pattern
- **High M (Overfitting)**: Model is too complex and fits noise along with the pattern

2.8.2 Visual Comparison

Degree	Behavior	Result
$M = 0$	Constant line	Severe underfitting
$M = 1$	Straight line	Underfitting
$M = 3$	Cubic curve	Good fit
$M = 9$	Passes through all points	Severe overfitting

2.8.3 The Overfitting Paradox

At $M = 9$ with $N = 10$ data points, the polynomial has 10 coefficients—exactly one per data point! This allows the curve to pass *exactly* through every training point, achieving $E(\mathbf{w}) \approx 0$.

Zero Training Error Does Not Mean Good Model!

A model with zero training error may have:

- Memorized the training data (including noise)
- Learned nothing about the true underlying pattern
- Very poor predictions on new data (high test error)

2.8.4 Coefficient Explosion: The Root Cause

When overfitting occurs, the coefficients w_j take **extreme values**:

Degree	w_0	w_1	w_2	...
$M = 0$	0.12	—	—	
$M = 1$	0.82	-1.27	—	
$M = 3$	0.31	7.99	-25.43	17.37
$M = 9$	0.35	232.37	-5321.83	... (extreme)

Large coefficients mean the curve oscillates wildly to pass through each point.

Why Doesn't Higher Degree "Include" Lower?

A natural question: Since a degree-9 polynomial can represent any degree-3 polynomial (by setting $w_4 = \cdots = w_9 = 0$), why doesn't it perform at least as well?
The answer: **The optimization finds the wrong solution.** With more parameters, there are many ways to achieve low training error, but most of them overfit. The optimization doesn't "know" to keep coefficients small.

2.9 Regularization: Controlling Complexity

2.9.1 The Key Insight

We identified the problem: **coefficients become too large**. The solution: **penalize large coefficients** directly in the error function.

2.9.2 Regularized Error Function

L2 Regularization (Ridge Regression)

$$\tilde{E}(\mathbf{w}) = \frac{1}{2}\sum_{n=1}^{N}(y(x_n;\mathbf{w}) - t_n)^2 + \frac{\lambda}{2}\|\mathbf{w}\|^2$$

where:

- $\|\mathbf{w}\|^2 = \sum_{j=0}^{M} w_j^2 = \mathbf{w}^\top\mathbf{w}$ is the squared L2 norm
- $\lambda \geq 0$ is the **regularization parameter**

2.9.3 The Tug-of-War

The regularized objective creates a balance:

Data Fit Term
$\sum(y-t)^2$
"Fit the data!"

$\longleftrightarrow$ Balance

Regularization Term
$\lambda\|\mathbf{w}\|^2$
"Keep weights small!"

- If $\lambda = 0$: No regularization, potential overfitting
- If λ is very large: Weights forced toward zero, potential underfitting
- Optimal λ: Balances fit quality with model simplicity

2.9.4 Effect of Regularization

With proper regularization, even a degree-9 polynomial can avoid overfitting:

- **Without regularization** ($\lambda = 0$): Wild oscillations, passing through all points

- **With regularization (λ optimal)**: Smooth curve approximating $\sin(2\pi x)$

Alternative Names

- **Ridge Regression**: Statistical term for L2 regularization
- **Weight Decay**: Neural network term (weights "decay" toward zero)
- **Shrinkage**: Coefficients are "shrunk" toward zero
- **Tikhonov Regularization**: Mathematical term

2.10 Hyperparameters and Validation

2.10.1 Parameters vs. Hyperparameters

Parameters and Hyperparameters

Parameters ($\mathbf{w}$):
- Learned from data during training
- Found by minimizing the error function

Hyperparameters (M, λ):
- Set *before* training
- Not learned from data (or learned separately)
- Control model structure and regularization

2.10.2 Why Not Optimize Hyperparameters Jointly?

A natural question: Why not minimize $\tilde{E}(\mathbf{w}, \lambda)$ jointly over $\mathbf{w}$ and λ?

Answer: The optimization would set $\lambda = 0$ and overfit! The regularization term fights against the objective, so joint optimization eliminates it.

2.10.3 Train, Validation, and Test Sets

To properly tune hyperparameters and evaluate the model:

Training (60-70%)	**Validation** (15-20	**Test** (15-20%)
Train model (learn $\mathbf{w}$)	Tune hyperparameters (M, λ)	Final evaluation (report results)

2.10.4 The Validation Process

1. **Choose** candidate hyperparameter values (e.g., $\lambda \in \{0.001, 0.01, 0.1, 1\}$)
2. **For each** candidate:
   (a) Train model on training set (find optimal $\mathbf{w}$)
   (b) Evaluate on validation set (compute validation error)
3. **Select** hyperparameters with lowest validation error
4. **Retrain** on training + validation data with best hyperparameters
5. **Report** final performance on test set

Never Train on Test Data!

The test set must remain completely unseen during model development. If you tune hyperparameters based on test performance, you're effectively training on the test set, and your reported performance will be overly optimistic.

2.10.5 Cross-Validation

When data is limited, we use **k-fold cross-validation**:

1. Divide data into k equal folds
2. For $i = 1, \ldots, k$:
   - Use fold i as validation, rest as training
   - Record validation error
3. Average the k validation errors

Common choices: $k = 5$ or $k = 10$.

2.11 Chapter Summary

- **Data generation**: Real-world data combines a pattern (signal) and noise. Our goal is to learn the pattern.
- **Curve fitting**: We fit a parameterized function (e.g., polynomial) to data by adjusting parameters $\mathbf{w}$.
- **Error function**: Quantifies fit quality. Sum of squared errors is common:

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^{N} (y(x_n; \mathbf{w}) - t_n)^2$$

- **Overfitting**: Occurs when the model is too complex. Signs include:
  - Very low training error but high test error
  - Extreme coefficient values
  - Curve oscillating wildly to pass through all points
- **Regularization**: Penalizes large coefficients to prevent overfitting:

$$\tilde{E}(\mathbf{w}) = E(\mathbf{w}) + \frac{\lambda}{2} \|\mathbf{w}\|^2$$

- **Hyperparameters**: M (polynomial degree) and λ (regularization strength) are set before training and tuned via validation.
- **Generalization**: The ultimate goal is good performance on unseen data, not just fitting training data.

Key Takeaways for Practice

1. More parameters $\neq$ better model (if unconstrained)
2. Always use a held-out test set for final evaluation
3. Regularization is essential when model capacity exceeds data size
4. Low training error is not sufficient; track validation error too
5. Hyperparameter tuning is as important as model selection

Chapter 3

From Biological to Artificial Neurons

3.1 Chapter Overview

This chapter traces the journey from biological neurons—the fundamental units of computation in the brain—to their artificial counterparts that form the foundation of neural networks. We explore the McCulloch-Pitts neuron, the perceptron, and understand both their capabilities and limitations.

Learning Objectives:

- Understand the structure and function of biological neurons
- Model neurons computationally using the McCulloch-Pitts model
- Understand the perceptron with weights and biases
- Implement logic gates (AND, OR) using neurons
- Recognize the concept of linear separability and its implications
- Understand why XOR cannot be solved by a single perceptron

3.2 The Biological Neuron

3.2.1 The Fundamental Unit of Neural Computation

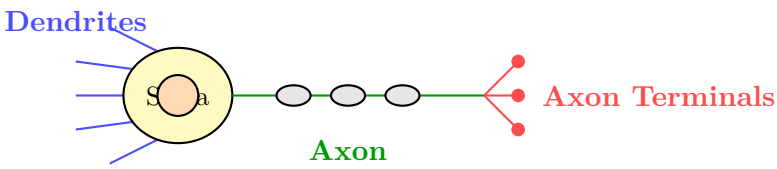
At the heart of all neural networks lies the **neuron**—inspired by the biological nerve cells that comprise our brains. The human brain contains approximately 86 billion neurons, interconnected in a vast network that enables thought, memory, and consciousness.

Biological Neuron

A **biological neuron** (or nerve cell) is the fundamental building block of the brain and nervous system. Neurons transmit information through electrochemical signals.

3.2.2 Structure of a Biological Neuron

A biological neuron consists of five key components:



1. **Dendrites**: Branch-like structures that *receive* signals from other neurons. They act as input channels.

2. **Cell Body (Soma)**: Contains the nucleus and *processes* incoming signals. Integrates all the information received from dendrites.
3. **Axon**: A long, cable-like structure that *transmits* electrical impulses away from the cell body toward other neurons.
4. **Axon Terminals**: End points of the axon where signals are passed to other neurons, typically through chemical messengers (neurotransmitters).
5. **Synapse**: The gap between one neuron's axon terminal and another neuron's dendrite—the site of communication between neurons.

How Neurons Communicate

Neurons communicate through two mechanisms:

1. **Electrical signals**: Travel along the axon as action potentials
2. **Chemical signals**: Neurotransmitters cross the synaptic gap to trigger the next neuron

When enough input signals arrive at a neuron, it "fires"—generating an electrical impulse that travels down the axon. This all-or-nothing firing is the basis for neural computation.

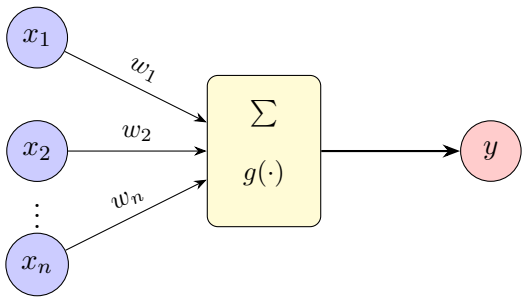
3.3 The Artificial Neuron: Computational Analog

3.3.1 From Biology to Computation

The key insight that enables artificial neural networks is that we can *abstract* the essential features of biological neurons into a mathematical model:

Biological Component	Computational Analog
Dendrites (receive input)	Input variables $x_1, x_2, \ldots, x_n$
Signal strength at synapse	Weights $w_1, w_2, \ldots, w_n$
Cell body (integration)	Summation function $\sum w_i x_i$
Firing threshold	Activation function $g(\cdot)$
Axon output	Output y

3.3.2 General Structure of an Artificial Neuron



Inputs Aggregation & Activation Output

The artificial neuron performs two operations:

1. **Aggregation**: Compute weighted sum of inputs: $z = \sum_{i=1}^{n} w_i x_i$
2. **Activation**: Apply a decision function: $y = g(z)$

3.4 The McCulloch-Pitts Neuron (1943)

3.4.1 The First Computational Neuron Model

In 1943, Warren McCulloch (neuroscientist) and Walter Pitts (logician) proposed the first mathematical model of a neuron. This model laid the groundwork for all subsequent neural network research.

McCulloch-Pitts (MP) Neuron

The MP neuron has the following characteristics:

- **Inputs**: Binary values $x_i \in \{0, 1\}$
- **Weights**: All equal to 1 (uniform weighting)
- **Aggregation**: Simple summation $\sum_{i=1}^{n} x_i$
- **Activation**: Threshold function

The neuron fires (outputs 1) if and only if:

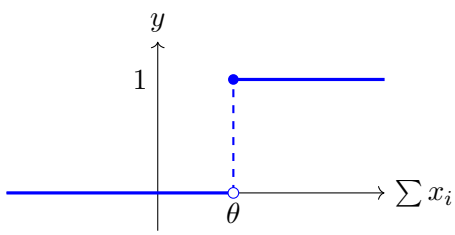
$$\sum_{i=1}^{n} x_i \geq \theta$$

where θ is the threshold.

3.4.2 Thresholding Logic

The activation function in MP neurons is a **step function**:

$$y = g\left(\sum_{i=1}^{n} x_i\right) = \begin{cases} 1 & \text{if } \sum_{i=1}^{n} x_i \geq \theta \\ 0 & \text{otherwise} \end{cases}$$



3.4.3 Inhibitory and Excitatory Inputs

McCulloch and Pitts recognized that biological neurons have two types of inputs:

- **Excitatory inputs**: Contribute positively toward reaching the threshold. They "encourage" the neuron to fire.
- **Inhibitory inputs**: Block the neuron from firing regardless of other inputs. If an inhibitory input is active, the neuron outputs 0.

Decision Example

Should I go to college today?
Inputs:

- x_1: Is my friend going? (excitatory)
- x_2: Is the weather nice? (excitatory)
- x_3: Do I have an exam? (excitatory)

- x_4: Am I seriously ill? (inhibitory)

If $x_4 = 1$ (ill), the output is 0 regardless of other inputs. Otherwise, if $x_1 + x_2 + x_3 \geq 2$, go to college.

3.5 Implementing Logic Gates

3.5.1 The Computational Foundation

Digital computers perform all computations using logic gates. If neurons can implement logic gates, they can potentially compute any function!

3.5.2 OR Gate

Truth Table:

x_1	x_2	OR
0	0	0
0	1	1
1	0	1
1	1	1

Implementation:

$$\text{Threshold: } \theta = 1$$

$$y = \begin{cases} 1 & \text{if } x_1 + x_2 \geq 1 \\ 0 & \text{otherwise} \end{cases}$$

3.5.3 AND Gate

Truth Table:

x_1	x_2	AND
0	0	0
0	1	0
1	0	0
1	1	1

Implementation:

$$\text{Threshold: } \theta = 2$$

$$y = \begin{cases} 1 & \text{if } x_1 + x_2 \geq 2 \\ 0 & \text{otherwise} \end{cases}$$

3.5.4 NOT Gate

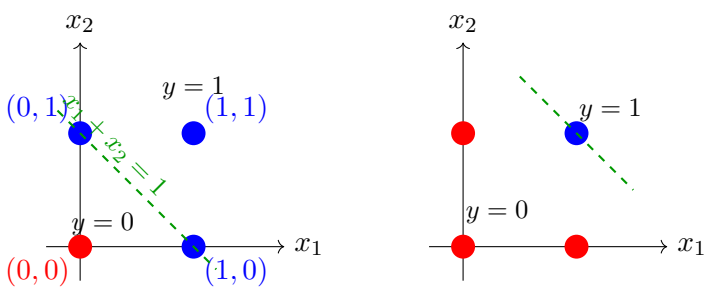
Using an inhibitory input:

$$y = \begin{cases} 1 & \text{if } x = 0 \\ 0 & \text{if } x = 1 \end{cases}$$

3.6 Geometric Interpretation: Linear Separability

3.6.1 Decision Boundaries

When we set a threshold condition like $x_1 + x_2 \geq \theta$, we're defining a **linear decision boundary** in the input space.



OR Gate **AND Gate**

Linear Separability

A dataset is **linearly separable** if there exists a hyperplane (a line in 2D, a plane in 3D) that perfectly separates the classes.
Mathematically, data points $\{(\mathbf{x}_i, y_i)\}$ are linearly separable if there exist weights $\mathbf{w}$ and bias b such that:

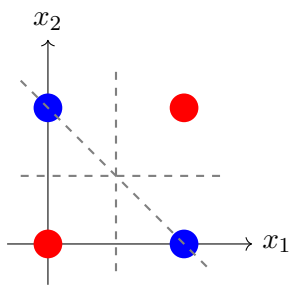
$$\mathbf{w}^\top \mathbf{x}_i + b > 0 \text{ for all } y_i = 1$$

$$\mathbf{w}^\top \mathbf{x}_i + b < 0 \text{ for all } y_i = 0$$

3.6.2 The XOR Problem

Consider the XOR (exclusive OR) function:

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0



XOR: NOT Linearly Separable

The XOR Problem

No single line can separate the blue points from the red points.
This means a single MP neuron (or a single perceptron) **cannot** compute the XOR function. This limitation, famously pointed out by Minsky and Papert (1969), temporarily halted neural network research.
The solution: **multiple layers** of neurons!

3.7 The Perceptron (1958)

3.7.1 Addressing MP Neuron Limitations

Frank Rosenblatt introduced the **perceptron** in 1958 to address three limitations of the MP neuron:

1. **Binary inputs only** → Allow real-valued inputs
2. **Equal weights** → Allow learnable weights
3. **Fixed threshold** → Replace with learnable bias

The Perceptron

The perceptron computes:

$$y = g\left(\sum_{i=1}^{n} w_i x_i + b\right) = g(\mathbf{w}^\top \mathbf{x} + b)$$

where:

- $x_i \in \mathbb{R}$: Real-valued inputs
- $w_i \in \mathbb{R}$: Learnable weights
- $b \in \mathbb{R}$: Learnable bias
- $g(\cdot)$: Step activation function

$$g(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

3.7.2 Understanding the Bias Term

The bias b serves as a learnable threshold. It controls how "eager" the neuron is to fire:

- **Positive bias**: Neuron tends to fire (biased toward positive output)
- **Negative bias**: Neuron is reluctant to fire (biased toward negative output)

Bias as Built-in Preference

Think of bias as the neuron's "default tendency."
Example: A neuron deciding whether to go to college:

- If you love college: High positive bias (you'll go unless many factors are negative)
- If you dislike college: Negative bias (you need many positive factors to go)

3.7.3 Parameters: What the Perceptron Learns

Component	Symbol	Type
Inputs	$x_1, \ldots, x_n$	Fixed (given)
Weights	$w_1, \ldots, w_n$	Learnable
Bias	b	Learnable
Output	y	Computed

Learning = Finding the right values for weights and bias from data.

3.8 The Perceptron Learning Algorithm

3.8.1 Learning from Data

Instead of manually setting weights, we want to **learn** them from labeled examples. Given training data $\{(\mathbf{x}_i, y_i)\}$, find $\mathbf{w}$ and b such that the perceptron correctly classifies all examples.

3.8.2 The Algorithm

1. **Initialize** weights $\mathbf{w}$ and bias b (e.g., to zeros or small random values)
2. **For each** training example $(\mathbf{x}, y)$:

(a) Compute prediction: $\hat{y} = g(\mathbf{w}^\top \mathbf{x} + b)$

(b) If $\hat{y} \neq y$ (misclassification):

$$\mathbf{w} \leftarrow \mathbf{w} + (y - \hat{y})\mathbf{x}$$

$$b \leftarrow b + (y - \hat{y})$$

3. **Repeat** until all examples are correctly classified (or max iterations)

Intuition Behind the Update Rule

- If $y = 1$ but $\hat{y} = 0$ (false negative):
  - We need to increase $\mathbf{w}^\top \mathbf{x} + b$
  - Update: $\mathbf{w} \leftarrow \mathbf{w} + \mathbf{x}$ (add the input to weights)
- If $y = 0$ but $\hat{y} = 1$ (false positive):
  - We need to decrease $\mathbf{w}^\top \mathbf{x} + b$
  - Update: $\mathbf{w} \leftarrow \mathbf{w} - \mathbf{x}$ (subtract the input)

3.8.3 Convergence Theorem

Perceptron Convergence Theorem

If the training data is **linearly separable**, the perceptron learning algorithm will converge to a solution (find separating weights) in a **finite number of steps**.

This is both good and bad news:

- **Good**: Guaranteed to find a solution if one exists
- **Bad**: If data is not linearly separable, the algorithm never converges

3.9 Chapter Summary

- **Biological neurons** are the brain's computational units, composed of dendrites (input), soma (processing), axon (transmission), and synapses (communication).
- **Artificial neurons** abstract biological neurons into mathematical models with inputs, weights, aggregation (summation), and activation (threshold).
- The **McCulloch-Pitts neuron** (1943) uses binary inputs, uniform weights, and thresholding logic. It can implement AND, OR, and NOT gates.
- **Linear separability**: A single neuron can only solve problems where classes can be separated by a hyperplane.
- The **XOR problem**: XOR is not linearly separable, so a single neuron cannot solve it. This motivates multi-layer networks.
- The **perceptron** (1958) adds real-valued inputs, learnable weights, and learnable bias, making learning from data possible.
- **Learning in neural networks** = adjusting weights and biases to minimize errors on training data.

Looking Ahead

The limitations of single neurons (especially the XOR problem) lead us toward:

1. **Multi-layer networks**: Stacking neurons to solve non-linearly separable problems
2. **Smooth activation functions**: Replacing the step function with differentiable functions like sigmoid, enabling gradient-based learning
3. **Backpropagation**: An algorithm to efficiently compute how to adjust weights in deep networks

Chapter 4

Activation Functions and Sigmoid Neurons

4.1 Chapter Overview

This chapter introduces the crucial concept of **activation functions**—the nonlinear transformations that give neural networks their power. We focus particularly on the sigmoid function, understand why smooth activation functions are preferable to step functions, and explore the forward pass through a neural network.

Learning Objectives:

- Understand why activation functions are necessary
- Compare step functions with smooth alternatives
- Master the sigmoid function and its properties
- Implement the forward pass through a network
- Compute loss functions (Mean Squared Error)

4.2 From Step Functions to Smooth Activations

4.2.1 Problems with the Step Function

Recall the perceptron's activation function:

$$g(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

This step function has significant drawbacks:

1. **Not differentiable**: The step function has an undefined derivative at $z = 0$ and zero derivative everywhere else. This prevents gradient-based learning.
2. **Non-smooth**: Small changes in weights can cause large jumps in output.
3. **No gradient signal**: When $z \neq 0$, the derivative is zero, providing no information about how to improve weights.

Why Differentiability Matters

Modern neural network training relies on **gradient descent**—an optimization algorithm that uses derivatives to find optimal weights. If the activation function isn't differentiable, we cannot compute how changes in weights affect the output, making learning impossible. The derivative tells us: "If I increase this weight by a tiny amount, how much will the

output change?" This information is essential for learning.

4.2.2 Desirable Properties of Activation Functions

An ideal activation function should be:

1. **Smooth (continuous)**: No jumps or discontinuities
2. **Differentiable**: Has a well-defined derivative everywhere (or almost everywhere)
3. **Nonlinear**: Without nonlinearity, neural networks collapse to linear models
4. **Bounded outputs**: Prevents exploding activations (for some functions)
5. **Computationally efficient**: Fast to compute

4.3 The Sigmoid Function

4.3.1 Definition

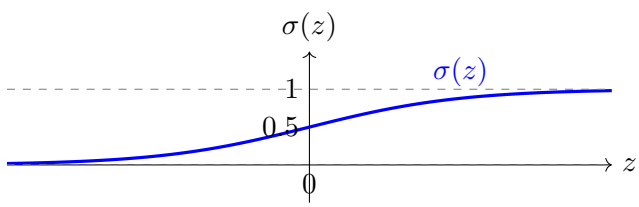
Sigmoid Function

The **sigmoid function** (also called the logistic function) is defined as:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

where $z = \mathbf{w}^\top \mathbf{x} + b$ (the weighted sum plus bias).

4.3.2 Properties of Sigmoid



Key Properties:

1. **Range**: $\sigma(z) \in (0, 1)$ for all $z \in \mathbb{R}$
2. **Sigmoid(0) = 0.5**: The function is centered at zero
3. **Monotonically increasing**: Larger inputs yield larger outputs
4. **S-shaped curve**: Smooth transition from 0 to 1
5. **Asymptotes**: $\lim_{z\to-\infty} \sigma(z) = 0$ and $\lim_{z\to\infty} \sigma(z) = 1$

4.3.3 Derivative of Sigmoid

A remarkable property of the sigmoid function is that its derivative has a simple form:

Sigmoid Derivative

$$\sigma'(z) = \sigma(z) \cdot (1 - \sigma(z))$$

Proof. Let $\sigma(z) = \frac{1}{1+e^{-z}} = (1 + e^{-z})^{-1}$.

Using the chain rule:

$$\begin{aligned}
\sigma'(z) &= -(1 + e^{-z})^{-2} \cdot (-e^{-z}) \\
&= \frac{e^{-z}}{(1 + e^{-z})^2} \\
&= \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}} \\
&= \sigma(z) \cdot \frac{e^{-z}}{1 + e^{-z}} \\
&= \sigma(z) \cdot \frac{1 + e^{-z} - 1}{1 + e^{-z}} \\
&= \sigma(z) \cdot \left(1 - \frac{1}{1 + e^{-z}}\right) \\
&= \sigma(z) \cdot (1 - \sigma(z))
\end{aligned}$$

□

This elegant form makes gradient computation efficient: once we compute $\sigma(z)$, the derivative is trivial to obtain.

4.3.4 Python Implementation

```python
import numpy as np

def sigmoid(z):
    """Compute the sigmoid of z."""
    return 1 / (1 + np.exp(-z))

def sigmoid_derivative(z):
    """Compute the derivative of sigmoid."""
    s = sigmoid(z)
    return s * (1 - s)
```

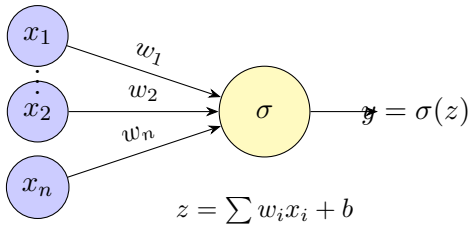
Listing 4.1: Sigmoid Function Implementation

4.4 The Sigmoid Neuron

4.4.1 Architecture

A **sigmoid neuron** replaces the step activation with the sigmoid function:

$$y = \sigma(\mathbf{w}^\top \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}}$$



4.4.2 Interpreting Sigmoid Output

The sigmoid output has a natural interpretation:

- Output ≈ 0: Strong confidence in class 0
- Output ≈ 0.5: Uncertain (at the decision boundary)
- Output ≈ 1: Strong confidence in class 1

This makes sigmoid ideal for **binary classification** problems.

4.5 Loss Functions

4.5.1 Why Loss Functions?

A loss function measures how "wrong" our predictions are. It quantifies the discrepancy between:

- $\hat{y}$ (predicted output)
- y (true label)

The goal of learning is to *minimize* the loss.

4.5.2 Mean Squared Error (MSE)

Mean Squared Error

For a dataset with N examples:

$$\mathcal{L}_{MSE} = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2$$

where $\hat{y}_i = \sigma(\mathbf{w}^\top \mathbf{x}_i + b)$ is the predicted value.

4.5.3 Properties of MSE

- **Non-negative**: Always ≥ 0
- **Zero at perfect prediction**: $\mathcal{L} = 0$ when $\hat{y}_i = y_i$ for all i
- **Penalizes large errors more**: Squaring amplifies large discrepancies
- **Differentiable**: Enables gradient-based optimization

```python
def mse_loss(y_pred, y_true):
    """Compute Mean Squared Error loss."""
    return np.mean((y_pred - y_true) ** 2)
```

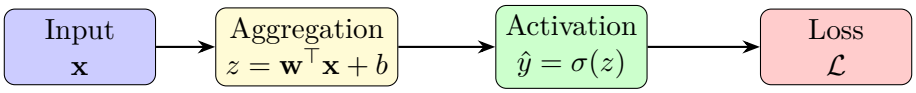
Listing 4.2: Mean Squared Error Implementation

4.6 The Forward Pass

4.6.1 Definition

The **forward pass** is the process of computing the output of a neural network given an input. Information flows from input to output through:

1. **Aggregation**: Compute $z = \mathbf{w}^\top \mathbf{x} + b$
2. **Activation**: Apply $\hat{y} = \sigma(z)$
3. **Loss computation**: Calculate $\mathcal{L}(\hat{y}, y)$



4.6.2 Example: Forward Pass by Hand

Consider three data points:

- $(x_1, y_1) = (0.5, 1)$
- $(x_2, y_2) = (2.0, 0)$
- $(x_3, y_3) = (-1.0, 1)$

With initial weights $w = -3.21$ and bias $b = 1.83$:

For point 1:

$$\begin{aligned} z_1 &= w \cdot x_1 + b = -3.21 \times 0.5 + 1.83 = 0.225 \\ \hat{y}_1 &= \sigma(0.225) = \frac{1}{1 + e^{-0.225}} \approx 0.556 \\ \mathcal{L}_1 &= (\hat{y}_1 - y_1)^2 = (0.556 - 1)^2 \approx 0.197 \end{aligned}$$

Repeat for all points and compute MSE:

$$\text{MSE} = \frac{0.197 + 0.0001 + 0.004}{3} \approx 0.067$$

4.7 Other Activation Functions

While sigmoid is historically important, modern networks use various activation functions:

4.7.1 Tanh (Hyperbolic Tangent)

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} = 2\sigma(2z) - 1$$

Range: $(-1, 1)$. Zero-centered, which can help learning.

4.7.2 ReLU (Rectified Linear Unit)

$$\text{ReLU}\left(()\, z\right) = \max(0, z)$$

Range: $[0, \infty)$. Simple, fast, and works well in practice. Currently the default choice for hidden layers.

4.7.3 Comparison

Function	Range	Pros	Cons
Sigmoid	$(0, 1)$	Probabilistic output	Vanishing gradient
Tanh	$(-1, 1)$	Zero-centered	Vanishing gradient
ReLU	$[0, \infty)$	Fast, no saturation	Dead neurons

4.8 Chapter Summary

- **Step functions** are problematic because they have zero gradients almost everywhere, making gradient-based learning impossible.
- **Sigmoid** $\sigma(z) = \frac{1}{1+e^{-z}}$ is smooth, differentiable, and maps any real number to $(0, 1)$.
- **Sigmoid's derivative** has the elegant form: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.
- **Loss functions** (like MSE) quantify prediction error and provide the objective to minimize during training.
- The **forward pass** computes predictions: input $\rightarrow$ aggregation $\rightarrow$ activation $\rightarrow$ output.
- Other activations (Tanh, ReLU) address sigmoid limitations and are commonly used in modern networks.

Chapter 5

Multi-Layer Perceptrons

5.1 Chapter Overview

This chapter introduces **Multi-Layer Perceptrons (MLPs)**—neural networks with multiple layers that can solve non-linearly separable problems. We establish formal definitions of network depth, layers, and notation that will be used throughout our study of deep learning.

Learning Objectives:

- Understand why multiple layers are needed
- Define depth and layers using graph theory
- Master standard neural network notation
- Trace information flow through deep networks
- Understand the matrix formulation of neural networks

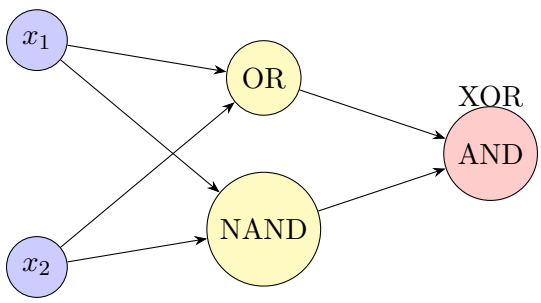
5.2 The Need for Multiple Layers

5.2.1 The XOR Revisited

Recall that a single perceptron cannot solve XOR because it's not linearly separable. However, XOR *can* be expressed as a combination of simpler functions:

$$\text{XOR}(x_1, x_2) = \text{AND}(\text{OR}(x_1, x_2), \text{NAND}(x_1, x_2))$$

This suggests that by **combining multiple neurons**, we can solve problems that individual neurons cannot.



Key Insight: Hierarchical Computation

Just as the visual cortex processes information in layers (edges → shapes → objects → scenes), neural networks can build complex representations by composing simpler ones layer by layer.

5.3 Neural Networks as Graphs

5.3.1 Directed Acyclic Graphs (DAGs)

Neural networks can be represented as **directed acyclic graphs (DAGs)**:

- **Nodes**: Neurons (computational units)
- **Edges**: Weighted connections (parameters)
- **Directed**: Information flows in one direction
- **Acyclic**: No loops or cycles

Source and Sink Nodes

- **Source nodes**: Nodes with no incoming edges (input layer)
- **Sink nodes**: Nodes with no outgoing edges (output layer)

5.3.2 Defining Depth

Depth of a Neural Network

The **depth** of a neural network is the number of edges on the *longest path* from any source node to any sink node.

$$\text{depth} = \max_{\text{paths } s \to t} (\text{number of edges on path})$$

where s is a source and t is a sink.

Computing Depth

Consider a network with:

- 3 input nodes (layer 0)
- 4 hidden nodes (layer 1)
- 4 hidden nodes (layer 2)
- 2 output nodes (layer 3)

The longest path has 3 edges (input $\to$ hidden1 $\to$ hidden2 $\to$ output).
Therefore, the network has **depth 3**.

5.3.3 What Makes a Network "Deep"?

Deep Network

A neural network is called **deep** if its depth is greater than 2.

$$\text{Deep network} \Leftrightarrow \text{depth} > 2$$

Networks with depth 2 (one hidden layer) are sometimes called "shallow" networks.

5.3.4 Defining Layers

Layer Assignment

A neuron belongs to **layer** k if the maximum number of edges from any source node to that neuron is k.

$$\text{layer}(n) = \max_{\text{paths } s \to n} (\text{number of edges})$$

This means:

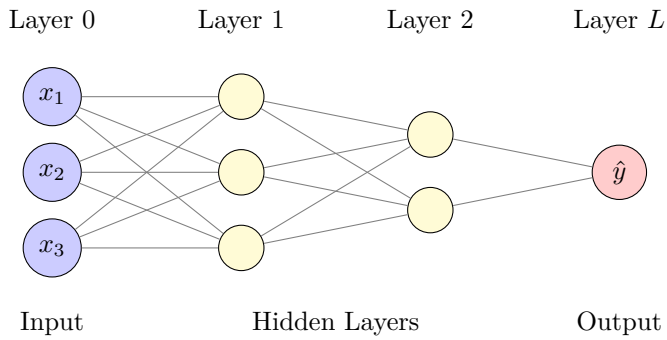
- Input nodes are in layer 0
- All neurons reachable in exactly k edges are in layer k
- Output nodes are in layer L (the final layer)

5.4 Standard Neural Network Notation

5.4.1 Network Structure

Consider a network with:

- L layers (not counting input)
- $n^{[l]}$ neurons in layer l



5.4.2 Weight Matrix Notation

Weight Matrix $\mathbf{W}^{[l]}$

The weight matrix $\mathbf{W}^{[l]}$ contains weights connecting layer $l-1$ to layer l:

$$\mathbf{W}^{[l]} \in \mathbb{R}^{n^{[l]} \times n^{[l-1]}}$$

Element $W_{ij}^{[l]}$ is the weight from neuron j in layer $l-1$ to neuron i in layer l.

Indexing convention: The destination neuron index comes first (row), then the source neuron index (column). This facilitates matrix multiplication in the forward pass.

5.4.3 Bias Vector Notation

Bias Vector $\mathbf{b}^{[l]}$

The bias vector for layer l:

$$\mathbf{b}^{[l]} \in \mathbb{R}^{n^{[l]}}$$

Element $b_i^{[l]}$ is the bias for neuron i in layer l.

5.4.4 Activation Notation

Pre-activation and Activation

For layer l:

- **Pre-activation** (before nonlinearity):

$$\mathbf{a}^{[l]} = \mathbf{W}^{[l]}\mathbf{h}^{[l-1]} + \mathbf{b}^{[l]}$$

- **Activation** (after nonlinearity):

$$\mathbf{h}^{[l]} = \sigma(\mathbf{a}^{[l]})$$

Note: $\mathbf{h}^{[0]} = \mathbf{x}$ (the input).

5.4.5 Summary of Notation

Symbol	Meaning	Dimensions
L	Number of layers (excluding input)	Scalar
$n^{[l]}$	Number of neurons in layer l	Scalar
$\mathbf{W}^{[l]}$	Weight matrix for layer l	$n^{[l]} \times n^{[l-1]}$
$\mathbf{b}^{[l]}$	Bias vector for layer l	$n^{[l]} \times 1$
$\mathbf{a}^{[l]}$	Pre-activation (linear output)	$n^{[l]} \times 1$
$\mathbf{h}^{[l]}$	Activation (hidden output)	$n^{[l]} \times 1$
$\mathbf{h}^{[0]} = \mathbf{x}$	Input	$n^{[0]} \times 1$
$\mathbf{h}^{[L]} = \hat{\mathbf{y}}$	Network output	$n^{[L]} \times 1$

5.5 Forward Propagation in MLPs

5.5.1 The Algorithm

Forward Propagation Algorithm

Input: $\mathbf{x}$ (input features), $\{\mathbf{W}^{[l]}, \mathbf{b}^{[l]}\}_{l=1}^{L}$ (parameters)
Output: $\hat{\mathbf{y}}$ (prediction)

1. Set $\mathbf{h}^{[0]} = \mathbf{x}$
2. For $l = 1, 2, \ldots, L$:
   (a) Compute pre-activation: $\mathbf{a}^{[l]} = \mathbf{W}^{[l]}\mathbf{h}^{[l-1]} + \mathbf{b}^{[l]}$
   (b) Apply activation: $\mathbf{h}^{[l]} = \sigma(\mathbf{a}^{[l]})$
3. Return $\hat{\mathbf{y}} = \mathbf{h}^{[L]}$

5.5.2 Matrix Formulation

For a batch of m examples, we can process all inputs simultaneously:

$$\mathbf{H}^{[0]} = \mathbf{X} \in \mathbb{R}^{n^{[0]} \times m}$$

$$\mathbf{A}^{[l]} = \mathbf{W}^{[l]}\mathbf{H}^{[l-1]} + \mathbf{b}^{[l]}\mathbf{1}^\top$$

$$\mathbf{H}^{[l]} = \sigma(\mathbf{A}^{[l]})$$

This matrix formulation enables efficient computation on GPUs.

5.5.3 Python Implementation

```python
import numpy as np

def forward_pass(X, weights, biases, activation=sigmoid):
    """
    Compute forward pass through an MLP.

    Args:
        X: Input data (n_features, n_samples)
        weights: List of weight matrices [W1, W2, ..., WL]
        biases: List of bias vectors [b1, b2, ..., bL]
        activation: Activation function

    Returns:
        List of activations [h0, h1, ..., hL]
    """
    H = [X]  # h0 = X

    for l in range(len(weights)):
        # Pre-activation
        A = weights[l] @ H[l] + biases[l]

        # Activation
        H.append(activation(A))

    return H
```

Listing 5.1: Forward Propagation

5.6 Representational Power

5.6.1 Universal Approximation Theorem

A profound result in neural network theory:

Universal Approximation Theorem (Informal)

A feedforward neural network with a single hidden layer containing a finite number of neurons can approximate any continuous function on compact subsets of $\mathbb{R}^n$, under mild

assumptions on the activation function.

This means MLPs are **universal function approximators**—they can theoretically represent any function we might want to learn.

Existence vs. Learnability

The Universal Approximation Theorem only guarantees *existence* of a good network. It says nothing about:

- How many neurons are needed
- Whether we can *find* the right weights through training
- Whether the solution will generalize to new data

In practice, deeper networks with fewer neurons per layer often work better than shallow, wide networks.

5.6.2 Why Depth Helps

Deep networks have advantages over shallow ones:

1. **Hierarchical features**: Each layer can learn increasingly abstract representations
2. **Efficiency**: Some functions require exponentially fewer neurons when represented with depth
3. **Modularity**: Lower layers can be reused for different tasks (transfer learning)

5.7 Chapter Summary

- **Multiple layers** enable solving non-linearly separable problems like XOR.
- Neural networks are **directed acyclic graphs (DAGs)** with source (input) and sink (output) nodes.
- **Depth** = number of edges on longest path from input to output. Deep networks have depth > 2.
- **Standard notation**:
  - $\mathbf{W}^{[l]}$: Weight matrix for layer l
  - $\mathbf{b}^{[l]}$: Bias vector for layer l
  - $\mathbf{a}^{[l]}$: Pre-activation, $\mathbf{h}^{[l]}$: Activation
- **Forward propagation**: Sequential computation through layers:

$$\mathbf{a}^{[l]} = \mathbf{W}^{[l]}\mathbf{h}^{[l-1]} + \mathbf{b}^{[l]}, \quad \mathbf{h}^{[l]} = \sigma(\mathbf{a}^{[l]})$$

- **Universal Approximation**: MLPs can approximate any continuous function, but finding good weights remains the challenge.

Looking Ahead

We now understand how information flows forward through a network. The next critical question is: **How do we learn the weights?**

This leads us to:

1. **Gradient Descent**: The optimization algorithm
2. **Backpropagation**: Efficient gradient computation

Chapter 6

Gradient Descent and Optimization

6.1 Chapter Overview

This chapter develops the mathematical foundations of learning: how to find optimal weights for neural networks. We formalize the learning problem as optimization, introduce the concept of **empirical risk minimization**, and develop **gradient descent**—the workhorse algorithm of deep learning.

Learning Objectives:

- Formalize learning as function approximation
- Understand empirical risk minimization (ERM)
- Master gradient descent and its variants
- Connect loss functions to divergence measures
- Implement gradient descent for neural networks

6.2 Formalizing the Learning Problem

6.2.1 The True Goal

When we train a neural network, we're trying to approximate an **unknown generating function** $g(\mathbf{x})$ that maps inputs to outputs. The true error between our model $f(\mathbf{x}; \mathbf{w})$ and the generating function would be:

$$\text{Total Error} = \int_{-\infty}^{\infty} D(f(\mathbf{x}; \mathbf{w}), g(\mathbf{x}))\, d\mathbf{x}$$

where $D(\cdot, \cdot)$ is a **divergence function** measuring the discrepancy.

6.2.2 Two Fundamental Problems

1. **We don't know** $g(\mathbf{x})$: The generating function is unknown—if we knew it, there would be no need for machine learning!
2. **Outputs can be noisy**: The same input might yield different outputs due to noise in the data generation process.

6.2.3 The Solution: Empirical Risk

Instead of minimizing true error (which is inaccessible), we minimize **empirical risk**—the average loss over our training data:

Empirical Risk

Given training data $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$:

$$R(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}(f(\mathbf{x}_i; \mathbf{w}), y_i)$$

where $\mathcal{L}$ is a loss function measuring prediction error.

Empirical Risk Minimization (ERM) is the principle of choosing parameters $\mathbf{w}^*$ that minimize empirical risk:

$$\mathbf{w}^* = \arg\min_{\mathbf{w}} R(\mathbf{w})$$

Why "Empirical"?

The term "empirical" means we're computing the risk from observed data (samples) rather than the true underlying distribution. This is the best we can do—we hope that if our samples are representative, the empirical risk is a good approximation to the true risk.

6.3 Divergence Functions and Losses

6.3.1 Properties of a Good Divergence

A divergence function $D(p, q)$ measures how different two distributions or functions are. It should satisfy:

1. **Non-negativity**: $D(p, q) \geq 0$ for all p, q
2. **Identity**: $D(p, q) = 0$ if and only if $p = q$

6.3.2 Common Loss Functions

Mean Squared Error (MSE)—for regression:

$$\mathcal{L}_{MSE}(\hat{y}, y) = (\hat{y} - y)^2$$

Cross-Entropy Loss—for classification:

$$\mathcal{L}_{CE}(\hat{y}, y) = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y})$$

We will explore these in detail when we discuss information theory.

6.4 Gradient Descent

6.4.1 The Optimization Problem

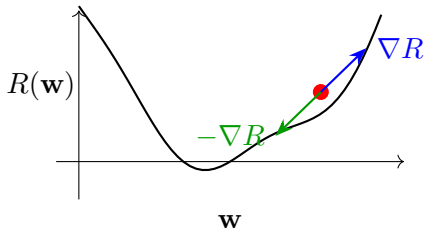
We want to find:

$$\mathbf{w}^* = \arg\min_{\mathbf{w}} R(\mathbf{w})$$

For neural networks, $R(\mathbf{w})$ is typically non-convex with many local minima. Gradient descent is an iterative algorithm that starts from an initial guess and moves "downhill" toward lower loss.

6.4.2 The Key Insight: Gradients Point Uphill

The **gradient** $\nabla_{\mathbf{w}} R(\mathbf{w})$ points in the direction of steepest *increase* of the loss. Therefore, to decrease the loss, we move in the *opposite* direction.



6.4.3 The Update Rule

Gradient Descent Update

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla_{\mathbf{w}} R(\mathbf{w}_t)$$

where:

- $\mathbf{w}_t$: Current parameters at iteration t
- $\eta > 0$: **Learning rate** (step size)
- $\nabla_{\mathbf{w}} R(\mathbf{w}_t)$: Gradient of loss w.r.t. parameters

6.4.4 The Gradient Descent Algorithm

Gradient Descent Algorithm

Input: Training data $\mathcal{D}$, learning rate η, iterations T

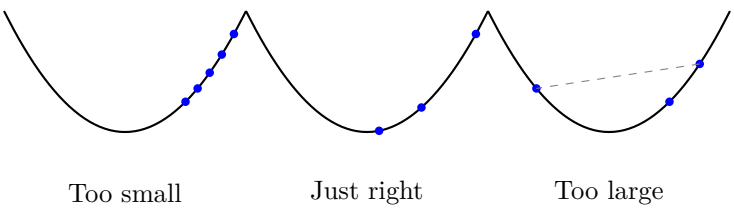
1. **Initialize** $\mathbf{w}_0$ randomly
2. **For** $t = 0, 1, \ldots, T-1$:
   (a) Compute loss: $R(\mathbf{w}_t) = \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}(f(\mathbf{x}_i; \mathbf{w}_t), y_i)$
   (b) Compute gradient: $\nabla_{\mathbf{w}} R(\mathbf{w}_t)$
   (c) Update: $\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla_{\mathbf{w}} R(\mathbf{w}_t)$
3. **Return** $\mathbf{w}_T$

6.5 The Learning Rate

6.5.1 Choosing the Learning Rate

The learning rate η is a critical hyperparameter:

If η is...	**Result**
Too small	Slow convergence, may get stuck
Too large	Overshooting, divergence
Just right	Steady convergence to minimum



6.6 Variants of Gradient Descent

6.6.1 Batch Gradient Descent

Uses the entire dataset to compute the gradient:

$$\nabla_{\mathbf{w}} R(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^{N} \nabla_{\mathbf{w}} \mathcal{L}(f(\mathbf{x}_i; \mathbf{w}), y_i)$$

Pros: Stable, accurate gradient. **Cons**: Slow for large datasets.

6.6.2 Stochastic Gradient Descent (SGD)

Uses a single random sample per update:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla_{\mathbf{w}} \mathcal{L}(f(\mathbf{x}_i; \mathbf{w}_t), y_i)$$

Pros: Fast updates. **Cons**: Noisy gradients, erratic convergence.

6.6.3 Mini-Batch Gradient Descent

Uses a small batch of B samples:

$$\nabla_{\mathbf{w}} R(\mathbf{w}) \approx \frac{1}{B} \sum_{i \in \text{batch}} \nabla_{\mathbf{w}} \mathcal{L}(f(\mathbf{x}_i; \mathbf{w}), y_i)$$

The standard choice in practice: balances efficiency and stability.

Typical Batch Sizes

Common mini-batch sizes are powers of 2 (32, 64, 128, 256) for computational efficiency on GPUs. Larger batches give more stable gradients; smaller batches provide regularization through noise.

6.7 Computing Gradients

6.7.1 The Challenge

For a neural network with millions of parameters, computing $\nabla_{\mathbf{w}} R(\mathbf{w})$ efficiently is non-trivial. We need:

$$\frac{\partial R}{\partial W_{ij}^{[l]}}, \quad \frac{\partial R}{\partial b_i^{[l]}} \quad \text{for all layers } l$$

6.7.2 The Solution: Backpropagation

Backpropagation is an efficient algorithm for computing gradients in neural networks. It uses the **chain rule** to propagate error signals backward through the network.

Key Insight

Backpropagation is not a learning algorithm—it's a method for efficiently computing gradients. The learning algorithm is gradient descent.

We will derive backpropagation in detail in the next chapter.

6.

```python
import numpy as np

def gradient_descent(X, y, model, loss_fn, learning_rate=0.01,
                     epochs=1000, batch_size=32):
    """
    Mini-batch gradient descent.

    Args:
        X: Training inputs (n_samples, n_features)
        y: Training targets (n_samples,)
        model: Neural network with forward() and backward() methods
        loss_fn: Loss function with gradient
        learning_rate: Step size
        epochs: Number of passes through data
        batch_size: Mini-batch size
    """
    n_samples = X.shape[0]
    losses = []

    for epoch in range(epochs):
        # Shuffle data
        indices = np.random.permutation(n_samples)
        X_shuffled = X[indices]
        y_shuffled = y[indices]

        epoch_loss = 0
        for i in range(0, n_samples, batch_size):
            # Get mini-batch
            X_batch = X_shuffled[i:i+batch_size]
            y_batch = y_shuffled[i:i+batch_size]

            # Forward pass
            y_pred = model.forward(X_batch)

            # Compute loss
            loss = loss_fn(y_pred, y_batch)
            epoch_loss += loss

            # Backward pass (compute gradients)
            gradients = model.backward(y_pred, y_batch)

            # Update parameters
            for param, grad in zip(model.parameters, gradients):
                param -= learning_rate * grad

        losses.append(epoch_loss / (n_samples // batch_size))

    return losses
```

Listing 6.1: Gradient Descent Implementation

6.9 Chapter Summary

- **Learning objective**: Minimize difference between model and true generating function. Since we don't know the true function, we minimize **empirical risk** over training data.
- **Empirical risk**: $R(\mathbf{w}) = \frac{1}{N}\sum_{i=1}^{N} \mathcal{L}(f(\mathbf{x}_i; \mathbf{w}), y_i)$
- **Gradient descent**: Iteratively update parameters in the direction opposite to the gradient:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla_{\mathbf{w}} R(\mathbf{w}_t)$$

- **Learning rate** η: Critical hyperparameter. Too small = slow; too large = diverge.
- **Variants**: Batch (full data), SGD (one sample), Mini-batch (best of both).
- **Backpropagation**: Efficient algorithm for computing gradients (next chapter).

Looking Ahead

We now understand the optimization objective and gradient descent. The remaining question is: **How do we efficiently compute gradients in deep networks?**
This leads to **backpropagation**—the subject of our next chapter.

Chapter 7

Backpropagation

Key Concept

Backpropagation is the algorithm at the **heart of deep learning**. It enables neural networks to learn by efficiently computing gradients of the loss function with respect to all parameters. The heart of backpropagation is the **chain rule** of calculus.

7.1 Motivation and Overview

When training a neural network, we have:

- A **loss function** $\mathcal{L}$ that measures how far our predictions are from the targets
- **Parameters** $\theta = \{W^{(1)}, b^{(1)}, W^{(2)}, b^{(2)}, \ldots, W^{(L)}, b^{(L)}\}$ that we can adjust
- The goal: adjust θ to minimize $\mathcal{L}$

To use gradient descent, we need $\frac{\partial \mathcal{L}}{\partial W^{(k)}}$ and $\frac{\partial \mathcal{L}}{\partial b^{(k)}}$ for all layers k. Backpropagation provides an efficient way to compute these gradients.

7.1.1 Independent vs. Dependent Parameters

Information

What Can We Actually Change? When inspecting a neural network, we can categorize quantities into:

- **Fixed inputs**: The data X is given; we cannot change it
- **Dependent parameters**: Hidden activations $h^{(k)}$, pre-activations $a^{(k)}$—these are *effects* that happen because of other causes
- **Independent parameters**: Weights $W^{(k)}$ and biases $b^{(k)}$—these are the *causes* that we have control over

The only quantities we can directly adjust are the **weights and biases**. Everything else is a consequence of these choices.

7.2 Review of Calculus Fundamentals

7.2.1 The Derivative as Rate of Change

For a univariate function $y = f(x)$, the derivative tells us how a small change in x affects y:

$$\frac{dy}{dx} = \lim_{\Delta x \to 0} \frac{f(x + \Delta x) - f(x)}{\Delta x}$$

If we nudge x by a small amount Δx, then:

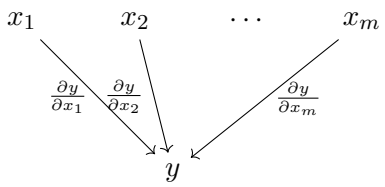
$$\Delta y \approx \frac{dy}{dx} \cdot \Delta x$$

7.2.2 Partial Derivatives and Multivariate Functions

For a function $y = f(x_1, x_2, \ldots, x_m)$ with multiple inputs, we use **partial derivatives**:

$$\frac{\partial y}{\partial x_i} = \text{rate of change of } y \text{ when only } x_i \text{ changes (others held constant)}$$

□Influence Diagram for Multivariate Functions



The **total change** in y when all inputs change:

$$\Delta y = \sum_{i=1}^{m} \frac{\partial y}{\partial x_i} \Delta x_i$$

7.2.3 The Chain Rule

For composed (nested) functions $y = f(g(x))$, we have:

$$\frac{dy}{dx} = \frac{dy}{dg} \cdot \frac{dg}{dx}$$

Derivation

Chain Rule for Nested Functions Let $y = f(g(x))$. If we nudge x by Δx:

1. First, g changes: $\Delta g = \frac{dg}{dx} \cdot \Delta x$
2. Then, y changes: $\Delta y = \frac{dy}{dg} \cdot \Delta g$
3. Substituting: $\Delta y = \frac{dy}{dg} \cdot \frac{dg}{dx} \cdot \Delta x$

Therefore: $\boxed{\frac{dy}{dx} = \frac{dy}{dg} \cdot \frac{dg}{dx}}$

7.2.4 Chain Rule with Multiple Paths

For $y = f(z_1, z_2, \ldots, z_m)$ where each $z_i = g_i(x)$:

$$\frac{\partial y}{\partial x} = \sum_{i=1}^{m} \frac{\partial y}{\partial z_i} \cdot \frac{\partial z_i}{\partial x}$$

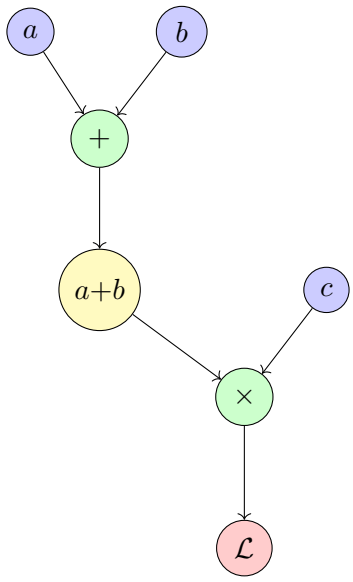
This is the **key insight for neural networks**: when one variable influences the output through multiple paths, we must **sum the contributions** from all paths.

7.3 Computational Graphs

A **computational graph** represents the computation as a directed acyclic graph (DAG) where:

- **Nodes** represent values (inputs, intermediate results, outputs)
- **Edges** represent operations that produce one value from others

Example 7.1. Consider $\mathcal{L} = (a + b) \times c$. The computational graph is:



To find $\frac{\partial \mathcal{L}}{\partial a}$, we trace the path from $\mathcal{L}$ back to a:

$$\frac{\partial \mathcal{L}}{\partial a} = \frac{\partial \mathcal{L}}{\partial (a+b)} \cdot \frac{\partial (a+b)}{\partial a} = c \cdot 1 = c$$

7.4 Forward Pass: Notation and Review

7.4.1 Notation Summary

For a neural network with L layers:

- n_k = number of neurons in layer k
- $W^{(k)} \in \mathbb{R}^{n_k \times n_{k-1}}$ = weight matrix connecting layer $k-1$ to layer k
- $b^{(k)} \in \mathbb{R}^{n_k}$ = bias vector for layer k
- $a^{(k)} \in \mathbb{R}^{n_k}$ = pre-activation (affine transformation) at layer k
- $h^{(k)} \in \mathbb{R}^{n_k}$ = post-activation (hidden output) at layer k
- f = activation function (e.g., sigmoid, ReLU)
- $h^{(0)} = X$ (input)
- $h^{(L)} = \hat{Y}$ (prediction)

7.4.2 Forward Pass Equations

Vector Form:

$$a^{(k)} = W^{(k)} h^{(k-1)} + b^{(k)} \tag{7.1}$$

$$h^{(k)} = f(a^{(k)}) \tag{7.2}$$

Element-wise Form:

$$a_i^{(k)} = \sum_{j=1}^{n_{k-1}} W_{ij}^{(k)} h_j^{(k-1)} + b_i^{(k)} \tag{7.3}$$

$$h_i^{(k)} = f(a_i^{(k)}) \tag{7.4}$$

where $W_{ij}^{(k)}$ is the weight connecting neuron j in layer $k-1$ to neuron i in layer k.

7.5 The Error (Delta) Vector

To simplify backpropagation calculations, we define the **error vector** (also called **delta** or **blame vector**):

Definition 7.1 (Error Vector)**.** The error at layer k is defined as:

$$\boxed{\delta^{(k)} = \frac{\partial \mathcal{L}}{\partial a^{(k)}}}$$

Element-wise:

$$\delta_i^{(k)} = \frac{\partial \mathcal{L}}{\partial a_i^{(k)}}$$

Interpretation: $\delta_i^{(k)}$ tells us how much neuron i in layer k is "to blame" for the loss. It quantifies how much a small change in the pre-activation $a_i^{(k)}$ would affect the loss.

7.6 Backpropagation Derivation

7.6.1 Step 1: Error at the Output Layer

At the output layer L, we compute the error using the chain rule:

Derivation

Output Layer Error $\delta^{(L)}$ We want $\delta^{(L)} = \frac{\partial \mathcal{L}}{\partial a^{(L)}}$.
By chain rule, since $\mathcal{L}$ depends on $a^{(L)}$ through $h^{(L)} = f(a^{(L)})$:

$$\delta_i^{(L)} = \frac{\partial \mathcal{L}}{\partial a_i^{(L)}} = \frac{\partial \mathcal{L}}{\partial h_i^{(L)}} \cdot \frac{\partial h_i^{(L)}}{\partial a_i^{(L)}}$$

Since $h_i^{(L)} = f(a_i^{(L)})$, we have $\frac{\partial h_i^{(L)}}{\partial a_i^{(L)}} = f'(a_i^{(L)})$.

Element-wise form:

$$\boxed{\delta_i^{(L)} = \frac{\partial \mathcal{L}}{\partial h_i^{(L)}} \cdot f'(a_i^{(L)})}$$

Vector form:

$$\boxed{\delta^{(L)} = \frac{\partial \mathcal{L}}{\partial h^{(L)}} \odot f'(a^{(L)})}$$

where $\odot$ denotes the Hadamard (element-wise) product.

Common Loss Functions

For **Mean Squared Error** loss $\mathcal{L} = \frac{1}{2}\|h^{(L)} - y\|^2$:

$$\frac{\partial \mathcal{L}}{\partial h^{(L)}} = h^{(L)} - y$$

For **Cross-Entropy with Softmax** output:

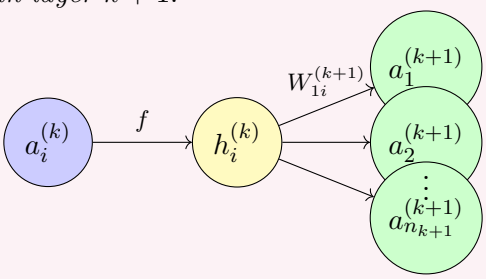
$$\delta^{(L)} = h^{(L)} - y \quad \text{(a remarkably simple result!)}$$

7.6.2 Step 2: Backpropagating the Error

For hidden layers $k = L-1, L-2, \ldots, 1$, we propagate the error backwards.

Derivation

Hidden Layer Error $\delta^{(k)}$ Consider neuron i in layer k. Its pre-activation $a_i^{(k)}$ affects the loss through *all neurons in layer* $k+1$:



By the chain rule with multiple paths:

$$\delta_i^{(k)} = \frac{\partial \mathcal{L}}{\partial a_i^{(k)}} \tag{7.5}$$

$$= \frac{\partial \mathcal{L}}{\partial h_i^{(k)}} \cdot \frac{\partial h_i^{(k)}}{\partial a_i^{(k)}} \tag{7.6}$$

$$= \left(\sum_{\ell=1}^{n_{k+1}} \frac{\partial \mathcal{L}}{\partial a_\ell^{(k+1)}} \cdot \frac{\partial a_\ell^{(k+1)}}{\partial h_i^{(k)}} \right) \cdot f'(a_i^{(k)}) \tag{7.7}$$

Since $a_\ell^{(k+1)} = \sum_j W_{\ell j}^{(k+1)} h_j^{(k)} + b_\ell^{(k+1)}$, we have:

$$\frac{\partial a_\ell^{(k+1)}}{\partial h_i^{(k)}} = W_{\ell i}^{(k+1)}$$

Therefore:

$$\boxed{\delta_i^{(k)} = f'(a_i^{(k)}) \cdot \sum_{\ell=1}^{n_{k+1}} W_{\ell i}^{(k+1)} \delta_\ell^{(k+1)}}$$

Vector form:

$$\boxed{\delta^{(k)} = \left(W^{(k+1)^\top} \delta^{(k+1)}\right) \odot f'(a^{(k)})}$$

7.6.3 Step 3: Gradients with Respect to Parameters

Now we compute the gradients with respect to weights and biases.

Derivation

Gradient with Respect to Weights Consider weight $W_{ij}^{(k)}$ connecting $h_j^{(k-1)}$ to $a_i^{(k)}$: Since $a_i^{(k)} = \sum_j W_{ij}^{(k)} h_j^{(k-1)} + b_i^{(k)}$:

$$\frac{\partial a_i^{(k)}}{\partial W_{ij}^{(k)}} = h_j^{(k-1)}$$

By chain rule:

$$\frac{\partial \mathcal{L}}{\partial W_{ij}^{(k)}} = \frac{\partial \mathcal{L}}{\partial a_i^{(k)}} \cdot \frac{\partial a_i^{(k)}}{\partial W_{ij}^{(k)}} = \delta_i^{(k)} \cdot h_j^{(k-1)}$$

Element-wise form:

$$\boxed{\frac{\partial \mathcal{L}}{\partial W_{ij}^{(k)}} = \delta_i^{(k)} \cdot h_j^{(k-1)}}$$

Matrix form:

$$\boxed{\frac{\partial \mathcal{L}}{\partial W^{(k)}} = \delta^{(k)} \cdot h^{(k-1)^\top}}$$

Derivation

Gradient with Respect to Biases Since $a_i^{(k)} = \sum_j W_{ij}^{(k)} h_j^{(k-1)} + b_i^{(k)}$:

$$\frac{\partial a_i^{(k)}}{\partial b_i^{(k)}} = 1$$

Therefore:

$$\boxed{\frac{\partial \mathcal{L}}{\partial b_i^{(k)}} = \delta_i^{(k)}}$$

Vector form:

$$\boxed{\frac{\partial \mathcal{L}}{\partial b^{(k)}} = \delta^{(k)}}$$

7.7 Summary: The Backpropagation Algorithm

Backpropagation

Input: Training example (X, y), network parameters $\{W^{(k)}, b^{(k)}\}_{k=1}^{L}$

Forward Pass:

1. Set $h^{(0)} = X$
2. For $k = 1, 2, \ldots, L$:
   - Compute $a^{(k)} = W^{(k)} h^{(k-1)} + b^{(k)}$ (cache $a^{(k)}$)
   - Compute $h^{(k)} = f(a^{(k)})$ (cache $h^{(k)}$)
3. Compute loss $\mathcal{L}(h^{(L)}, y)$

Backward Pass:

1. Compute output error: $\delta^{(L)} = \frac{\partial \mathcal{L}}{\partial h^{(L)}} \odot f'(a^{(L)})$
2. For $k = L-1, L-2, \ldots, 1$:
   - Compute $\delta^{(k)} = \left(W^{(k+1)^\top} \delta^{(k+1)}\right) \odot f'(a^{(k)})$
3. Compute gradients for all layers:
   $\frac{\partial \mathcal{L}}{\partial W^{(k)}} = \delta^{(k)} \cdot h^{(k-1)^\top}$ and $\frac{\partial \mathcal{L}}{\partial b^{(k)}} = \delta^{(k)}$

7.8 Worked Example: Backpropagation by Hand

Let us perform one complete forward and backward pass on a small network.

7.8.1 Network Setup

- **Architecture**: 2 inputs → 2 hidden → 2 hidden → 2 outputs
- **Activation**: Sigmoid throughout
- **Loss**: Mean Squared Error

Weight Matrices:

$$W^{(1)} = \begin{pmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \end{pmatrix}, \quad W^{(2)} = \begin{pmatrix} 0.5 & 0.6 \\ 0.7 & 0.8 \end{pmatrix}, \quad W^{(3)} = \begin{pmatrix} 0.9 & 1.0 \\ 1.1 & 1.2 \end{pmatrix}$$

Bias Vectors:

$$b^{(1)} = b^{(2)} = b^{(3)} = \begin{pmatrix} 0.1 \\ 0.1 \end{pmatrix}$$

Training Example:

$$X = h^{(0)} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \qquad y = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

7.8.2 Forward Pass

Layer 1:

$$a^{(1)} = W^{(1)} h^{(0)} + b^{(1)} = \begin{pmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} + \begin{pmatrix} 0.1 \\ 0.1 \end{pmatrix} = \begin{pmatrix} 0.2 \\ 0.4 \end{pmatrix} \tag{7.8}$$

$$h^{(1)} = \sigma(a^{(1)}) = \begin{pmatrix} \sigma(0.2) \\ \sigma(0.4) \end{pmatrix} = \begin{pmatrix} 0.550 \\ 0.599 \end{pmatrix} \tag{7.9}$$

Layer 2:

$$a^{(2)} = W^{(2)}h^{(1)} + b^{(2)} = \begin{pmatrix} 0.5(0.550) + 0.6(0.599) + 0.1 \\ 0.7(0.550) + 0.8(0.599) + 0.1 \end{pmatrix} = \begin{pmatrix} 0.734 \\ 0.964 \end{pmatrix} \tag{7.10}$$

$$h^{(2)} = \sigma(a^{(2)}) = \begin{pmatrix} 0.676 \\ 0.724 \end{pmatrix} \tag{7.11}$$

Layer 3 (Output):

$$a^{(3)} = W^{(3)}h^{(2)} + b^{(3)} = \begin{pmatrix} 0.9(0.676) + 1.0(0.724) + 0.1 \\ 1.1(0.676) + 1.2(0.724) + 0.1 \end{pmatrix} = \begin{pmatrix} 1.432 \\ 1.712 \end{pmatrix} \tag{7.12}$$

$$h^{(3)} = \sigma(a^{(3)}) = \begin{pmatrix} 0.807 \\ 0.847 \end{pmatrix} \tag{7.13}$$

Loss:

$$\mathcal{L} = \frac{1}{2}\|h^{(3)} - y\|^2 = \frac{1}{2}\left[(0.807 - 0)^2 + (0.847 - 1)^2\right] = \frac{1}{2}(0.651 + 0.023) = 0.337$$

7.8.3 Backward Pass

Output Layer Error $\delta^{(3)}$:

For MSE loss and sigmoid activation:

$$\delta^{(3)} = (h^{(3)} - y) \odot \sigma'(a^{(3)}) \tag{7.14}$$

$$= \begin{pmatrix} 0.807 - 0 \\ 0.847 - 1 \end{pmatrix} \odot \begin{pmatrix} \sigma(1.432)(1 - \sigma(1.432)) \\ \sigma(1.712)(1 - \sigma(1.712)) \end{pmatrix} \tag{7.15}$$

$$= \begin{pmatrix} 0.807 \\ -0.153 \end{pmatrix} \odot \begin{pmatrix} 0.156 \\ 0.130 \end{pmatrix} = \begin{pmatrix} 0.126 \\ -0.020 \end{pmatrix} \tag{7.16}$$

Hidden Layer Error $\delta^{(2)}$:

$$\delta^{(2)} = \left(W^{(3)^\top}\delta^{(3)}\right) \odot \sigma'(a^{(2)}) \tag{7.17}$$

$$= \begin{pmatrix} 0.9 & 1.1 \\ 1.0 & 1.2 \end{pmatrix} \begin{pmatrix} 0.126 \\ -0.020 \end{pmatrix} \odot \begin{pmatrix} 0.219 \\ 0.200 \end{pmatrix} \tag{7.18}$$

$$= \begin{pmatrix} 0.091 \\ 0.102 \end{pmatrix} \odot \begin{pmatrix} 0.219 \\ 0.200 \end{pmatrix} = \begin{pmatrix} 0.020 \\ 0.020 \end{pmatrix} \tag{7.19}$$

Hidden Layer Error $\delta^{(1)}$:

$$\delta^{(1)} = \left(W^{(2)^\top}\delta^{(2)}\right) \odot \sigma'(a^{(1)}) \tag{7.20}$$

$$\approx \begin{pmatrix} 0.006 \\ 0.007 \end{pmatrix} \tag{7.21}$$

Key Observation: The errors **dilute as they propagate backwards**. This is why deep networks can suffer from the *vanishing gradient problem.*

7.8.4 Parameter Gradients

Gradients for $W^{(3)}$:

$$\frac{\partial \mathcal{L}}{\partial W^{(3)}} = \delta^{(3)} \cdot h^{(2)^\top} = \begin{pmatrix} 0.126 \\ -0.020 \end{pmatrix} \begin{pmatrix} 0.676 & 0.724 \end{pmatrix} = \begin{pmatrix} 0.085 & 0.091 \\ -0.014 & -0.015 \end{pmatrix}$$

Gradients for $b^{(3)}$:

$$\frac{\partial \mathcal{L}}{\partial b^{(3)}} = \delta^{(3)} = \begin{pmatrix} 0.126 \\ -0.020 \end{pmatrix}$$

7.8.5 Gradient Descent Update

With learning rate $\eta = 0.1$:

$$W^{(3)}_{\text{new}} = W^{(3)} - \eta \cdot \frac{\partial \mathcal{L}}{\partial W^{(3)}} \tag{7.22}$$

$$= \begin{pmatrix} 0.9 & 1.0 \\ 1.1 & 1.2 \end{pmatrix} - 0.1 \cdot \begin{pmatrix} 0.085 & 0.091 \\ -0.014 & -0.015 \end{pmatrix} \tag{7.23}$$

$$= \begin{pmatrix} 0.891 & 0.991 \\ 1.101 & 1.202 \end{pmatrix} \tag{7.24}$$

7.9 Mini-batch Gradient Descent

In practice, we don't update after each example. Instead:

1. **Shuffle** the dataset
2. **Divide** into mini-batches of size m (e.g., 32, 64, 128)
3. For each mini-batch:
   - Compute gradients for each example
   - **Average** the gradients across the batch:

$$\frac{\partial \mathcal{L}}{\partial W^{(k)}} = \frac{1}{m} \sum_{i=1}^{m} \frac{\partial \mathcal{L}_i}{\partial W^{(k)}}$$

   - Update parameters using the averaged gradient

Information

Why Mini-batches?

- Computing gradients for all examples (batch gradient descent) is too slow
- Single-example updates (stochastic gradient descent) are too noisy
- Mini-batches provide a **balance**: reasonably accurate gradient estimates with computational efficiency
- The "stochastic" navigation of the loss landscape can help escape local minima

7.10 Why is it Called "Backpropagation"?

The name comes from the fact that:

1. Errors are computed starting from the **output layer**

2. Errors are **propagated backwards** through the network

3. To compute $\delta^{(k)}$, we need $\delta^{(k+1)}$, which comes "after" layer k in the forward direction

Key Concept

Backpropagation is an efficient implementation of the chain rule. It computes all gradients in $O(n)$ time (where n is the number of parameters) by:
1. **Caching** intermediate values during the forward pass
2. **Reusing** error computations during the backward pass

Without this caching, naive gradient computation would be exponentially expensive.

7.11 Sigmoid Derivative

For the sigmoid function $\sigma(x) = \frac{1}{1+e^{-x}}$, the derivative has a beautiful form:

Derivation

Sigmoid Derivative

$$\begin{aligned}
\sigma'(x) &= \frac{d}{dx}\left(\frac{1}{1+e^{-x}}\right) && (7.25)\\
&= \frac{e^{-x}}{(1+e^{-x})^2} && (7.26)\\
&= \frac{1}{1+e^{-x}} \cdot \frac{e^{-x}}{1+e^{-x}} && (7.27)\\
&= \sigma(x) \cdot \frac{(1+e^{-x})-1}{1+e^{-x}} && (7.28)\\
&= \sigma(x) \cdot (1-\sigma(x)) && (7.29)
\end{aligned}$$

$$\boxed{\sigma'(x) = \sigma(x)(1-\sigma(x))}$$

This is computationally convenient because if we've already computed $\sigma(x)$ during the forward pass, we can compute the derivative without any additional exponentiations!

7.12 Key Takeaways

Summary

Core Ideas:
- Backpropagation efficiently computes $\frac{\partial \mathcal{L}}{\partial W^{(k)}}$ and $\frac{\partial \mathcal{L}}{\partial b^{(k)}}$ for all layers
- The heart of backpropagation is the **chain rule** of calculus
- We define the **error vector** $\delta^{(k)} = \frac{\partial \mathcal{L}}{\partial a^{(k)}}$ to simplify calculations

Key Equations:

$$\delta^{(L)} = \frac{\partial \mathcal{L}}{\partial h^{(L)}} \odot f'(a^{(L)}) \qquad \text{(output layer)} \tag{7.30}$$

$$\delta^{(k)} = \left(W^{(k+1)^\top} \delta^{(k+1)}\right) \odot f'(a^{(k)}) \qquad \text{(hidden layers)} \tag{7.31}$$

$$\frac{\partial \mathcal{L}}{\partial W^{(k)}} = \delta^{(k)} \cdot h^{(k-1)^\top} \qquad \text{(weight gradients)} \tag{7.32}$$

$$\frac{\partial \mathcal{L}}{\partial b^{(k)}} = \delta^{(k)} \qquad \text{(bias gradients)} \tag{7.33}$$

Chapter 8

Information Theory and Loss Functions

Key Concept

Information theory provides the mathematical foundation for understanding how to measure the "distance" between probability distributions. This leads to principled loss functions like **cross-entropy**, which are fundamental to training neural networks for classification.

8.1 The Learning Problem

8.1.1 Setup

We have:

- An unknown true function $g : \mathcal{X} \to \mathcal{Y}$
- A dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$ sampled from some distribution
- A model f_θ parameterized by θ
- Goal: Find θ such that $f_\theta(x) \approx g(x)$ for all x

8.1.2 The Divergence Problem

To measure how well our model f_θ approximates the true function g, we need a **divergence function**:

$$D(f_\theta, g) : \text{measures "distance" between } f_\theta \text{ and } g$$

Problem: We don't know g! We only have samples (x, y) from it.

Solution: We use the **empirical distribution** from our data as a proxy for the true distribution.

8.2 Information and Surprise

8.2.1 Intuition: Data as Surprise

Information

The Core Insight When we receive data, we gain **information**. Information is inversely related to **probability**—the less likely an event, the more surprising (informative) it is when it occurs.

Consider tossing a fair coin:

- Getting heads has probability $\frac{1}{2}$
- Getting heads 10 times in a row has probability $\left(\frac{1}{2}\right)^{10} = \frac{1}{1024}$

The second event is much more "surprising" and thus carries more information.

8.2.2 Defining Information

We want a function $I(p)$ that measures the "information content" of an event with probability p:

Desirable Properties:

1. $I(1) = 0$ (certain events provide no information)
2. $I(p)$ should be decreasing in p (rare events are more informative)
3. $I(p_1 \cdot p_2) = I(p_1) + I(p_2)$ (information from independent events should add)

Derivation

Information Content From property 3, we need $I(p_1 \cdot p_2) = I(p_1) + I(p_2)$.
The logarithm is the only function satisfying this:

$$\log(p_1 \cdot p_2) = \log(p_1) + \log(p_2)$$

But $\log(p)$ is negative for $p < 1$, and we want positive information. So:

$$\boxed{I(p) = -\log(p) = \log\left(\frac{1}{p}\right)}$$

For base 2, information is measured in **bits**.

Examples:

- Fair coin flip: $I(0.5) = -\log_2(0.5) = 1$ bit
- Rolling a 6 on a fair die: $I(1/6) = -\log_2(1/6) \approx 2.58$ bits
- Rolling 10 sixes in a row: $I((1/6)^{10}) \approx 25.8$ bits

8.3 Entropy

8.3.1 Definition

Entropy measures the *expected* (average) information content of a random variable.

Definition 8.1 (Entropy)**.** For a discrete random variable X with probability distribution $P(X)$:

$$\boxed{H(X) = -\sum_x P(x) \log P(x) = \mathbb{E}_{x \sim P}[-\log P(x)]}$$

This is the **expected surprise** when observing samples from distribution P.

8.3.2 Properties of Entropy

- $H(X) \geq 0$ (entropy is non-negative)
- $H(X) = 0$ if and only if X is deterministic (one outcome has probability 1)

- For n equally likely outcomes: $H(X) = \log n$ (maximum entropy)

8.3.3 Examples

Fair coin ($p = 0.5$):

$$H = -[0.5 \log_2(0.5) + 0.5 \log_2(0.5)] \tag{8.1}$$
$$= -[0.5 \cdot (-1) + 0.5 \cdot (-1)] = 1 \text{ bit} \tag{8.2}$$

Biased coin ($p = 0.9$ for heads):

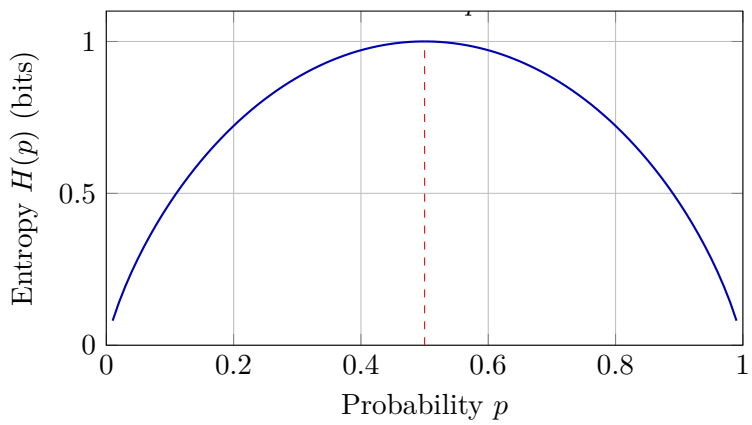
$$H = -[0.9 \log_2(0.9) + 0.1 \log_2(0.1)] \tag{8.3}$$
$$\approx -[0.9 \cdot (-0.152) + 0.1 \cdot (-3.322)] \tag{8.4}$$
$$\approx 0.469 \text{ bits} \tag{8.5}$$

The biased coin has **lower entropy**—its outcomes are more predictable!

8.3.4 Entropy Visualization

For a binary random variable with $P(\text{heads}) = p$:

$$H(p) = -p \log_2(p) - (1-p) \log_2(1-p)$$



8.4 Cross-Entropy

8.4.1 Motivation

Suppose we have:

- True distribution P (the actual data distribution)
- Approximate distribution Q (our model's predictions)

Cross-entropy measures the expected information content when we use distribution Q to encode events that actually come from distribution P.

Definition 8.2 (Cross-Entropy)**.**

$$\boxed{H(P, Q) = -\sum_x P(x) \log Q(x) = \mathbb{E}_{x \sim P}[-\log Q(x)]}$$

Interpretation: If the true distribution is P but we encode using Q, cross-entropy tells us the average number of bits needed per message.

8.4.2 Key Insight

Theorem 8.1 (Cross-Entropy and Entropy)**.**

$$H(P, Q) \geq H(P)$$

with equality if and only if $Q = P$.

This tells us:

- Using the wrong distribution $Q \neq P$ always requires more bits on average
- Cross-entropy is minimized when $Q = P$
- Therefore, minimizing cross-entropy pushes Q toward P!

8.5 KL Divergence

8.5.1 Definition

The **Kullback-Leibler (KL) divergence** measures the "extra bits" needed when using Q instead of P:

Definition 8.3 (KL Divergence)**.**

$$\boxed{D_{KL}(P\|Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)} = H(P, Q) - H(P)}$$

Interpretation: KL divergence is the "waste" in using Q to encode data from P.

8.5.2 Properties

- $D_{KL}(P\|Q) \geq 0$ (Gibbs' inequality)
- $D_{KL}(P\|Q) = 0$ if and only if $P = Q$
- **Not symmetric**: $D_{KL}(P\|Q) \neq D_{KL}(Q\|P)$ in general
- Not a true metric (doesn't satisfy triangle inequality)

8.5.3 Why Cross-Entropy for Loss?

In minimizing loss, we want Q_θ (our model) to match P (true distribution):

$$\min_\theta D_{KL}(P\|Q_\theta) = \min_\theta [H(P, Q_\theta) - H(P)]$$

Since $H(P)$ doesn't depend on θ:

$$\min_\theta D_{KL}(P\|Q_\theta) = \min_\theta H(P, Q_\theta)$$

Minimizing cross-entropy is equivalent to minimizing KL divergence!
This is why cross-entropy is the natural loss function for classification.

8.6 The Softmax Function

For multi-class classification, we need to convert network outputs into a probability distribution. The **softmax function** accomplishes this.

Definition 8.4 (Softmax)**.** For a vector $z = (z_1, z_2, \ldots, z_K)$ of logits:

$$\boxed{\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}}$$

8.6.1 Properties

1. **Valid probability distribution**: All outputs are positive and sum to 1

$$\sum_{i=1}^{K} \text{softmax}(z)_i = 1$$

2. **Preserves ordering**: If $z_i > z_j$, then $\text{softmax}(z)_i > \text{softmax}(z)_j$
3. **Differentiable**: Can be used with gradient-based optimization
4. **Shift invariance**: $\text{softmax}(z + c) = \text{softmax}(z)$ for any constant c

8.6.2 Example

For $z = (2, 1, 0)$:

$$\begin{aligned}
\text{softmax}(z) &= \frac{1}{e^2 + e^1 + e^0}(e^2, e^1, e^0) && (8.6)\\
&= \frac{1}{7.389 + 2.718 + 1}(7.389, 2.718, 1) && (8.7)\\
&\approx (0.665, 0.245, 0.090) && (8.8)
\end{aligned}$$

8.6.3 Softmax Temperature

A generalization introduces **temperature** T:

$$\text{softmax}(z/T)_i = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

- $T \to 0$: Outputs become one-hot (winner-take-all)
- $T = 1$: Standard softmax
- $T \to \infty$: Outputs become uniform

8.7 Cross-Entropy Loss for Classification

8.7.1 Binary Classification: Binary Cross-Entropy (BCE)

For binary classification with true label $y \in \{0, 1\}$ and predicted probability $\hat{y} = P(Y = 1|X)$:

Definition 8.5 (Binary Cross-Entropy Loss)**.**

$$\boxed{\mathcal{L}_{BCE} = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]}$$

Analysis by cases:

- If $y = 1$: $\mathcal{L} = -\log(\hat{y})$ (penalizes low $\hat{y}$)
- If $y = 0$: $\mathcal{L} = -\log(1 - \hat{y})$ (penalizes high $\hat{y}$)

8.7.2 Multi-class Classification: Categorical Cross-Entropy (CCE)

For K classes with one-hot encoded true label $y = (y_1, \ldots, y_K)$ and predicted probabilities $\hat{y} = (\hat{y}_1, \ldots, \hat{y}_K)$:

Definition 8.6 (Categorical Cross-Entropy Loss)**.**

$$\boxed{\mathcal{L}_{CCE} = -\sum_{k=1}^{K} y_k \log(\hat{y}_k)}$$

Since y is one-hot (only one $y_c = 1$, others are 0):

$$\mathcal{L}_{CCE} = -\log(\hat{y}_c)$$

where c is the true class.

Interpretation: We only penalize the predicted probability of the *correct* class.

8.7.3 Example: Multi-class Classification

Scenario: 3-class classification, true class is 2.

True label (one-hot): $y = (0, 1, 0)$

Predicted probabilities: $\hat{y} = (0.1, 0.7, 0.2)$

$$\mathcal{L} = -[\underbrace{0 \cdot \log(0.1)}_{=0} + \underbrace{1 \cdot \log(0.7)}_{=-0.357} + \underbrace{0 \cdot \log(0.2)}_{=0}] = 0.357$$

If prediction were perfect ($\hat{y} = (0, 1, 0)$): $\mathcal{L} = -\log(1) = 0$.

8.8 Maximum Likelihood Estimation Perspective

Cross-entropy loss emerges naturally from the **maximum likelihood estimation** (MLE) framework.

8.8.1 The Four-Step Recipe for Loss Construction

Constructing Loss Functions via MLE

1. **Choose a probability distribution** for the output Y
2. **Parameterize the distribution** with the neural network output
3. **Compute negative log-likelihood** (this is your loss)
4. **Find argmax** for inference (point prediction)

8.8.2 Deriving Binary Cross-Entropy

Step 1: Model output as Bernoulli distribution

$$P(Y = y|X = x) = \hat{y}^y(1 - \hat{y})^{1-y}$$

where $\hat{y} = \sigma(\theta^\top x)$ (sigmoid output).

Step 2: Likelihood for N i.i.d. samples

$$\mathcal{L}(\theta) = \prod_{i=1}^{N} P(y_i|x_i;\theta) = \prod_{i=1}^{N} \hat{y}_i^{y_i}(1 - \hat{y}_i)^{1-y_i}$$

Step 3: Negative log-likelihood

$$-\log \mathcal{L}(\theta) = -\sum_{i=1}^{N} [y_i \log \hat{y}_i + (1 - y_i)\log(1 - \hat{y}_i)] \tag{8.9}$$

$$= \sum_{i=1}^{N} \mathcal{L}_{BCE}(y_i, \hat{y}_i) \tag{8.10}$$

Binary cross-entropy is simply the negative log-likelihood of the Bernoulli distribution!

8.8.3 Deriving Categorical Cross-Entropy

Step 1: Model output as Categorical distribution

$$P(Y = k|X) = \hat{y}_k = \text{softmax}(z)_k$$

Step 2: Likelihood (one-hot representation)

$$P(Y = y|X) = \prod_{k=1}^{K} \hat{y}_k^{y_k}$$

Step 3: Negative log-likelihood

$$-\log P(Y = y|X) = -\sum_{k=1}^{K} y_k \log \hat{y}_k = \mathcal{L}_{CCE}$$

8.8.4 MSE from Gaussian Assumption

Step 1: Assume Gaussian noise

$$P(y|x) = \mathcal{N}(y|\hat{y}, \sigma^2)$$

Step 2: Negative log-likelihood

$$-\log P(y|x) = -\log \left[\frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y - \hat{y})^2}{2\sigma^2} \right) \right] \tag{8.11}$$

$$= \frac{(y - \hat{y})^2}{2\sigma^2} + \text{const} \tag{8.12}$$

Mean Squared Error is the negative log-likelihood of a Gaussian distribution!

8.9 Gradient of Cross-Entropy with Softmax

A remarkable property emerges when combining softmax with cross-entropy loss.

Derivation

Softmax + Cross-Entropy Gradient Let z be the logits (pre-softmax), $\hat{y} = \text{softmax}(z)$, and $\mathcal{L} = -\sum_k y_k \log \hat{y}_k$.
For the gradient $\frac{\partial \mathcal{L}}{\partial z_i}$:
Case 1: $i = c$ (correct class)

$$\frac{\partial \mathcal{L}}{\partial z_c} = \hat{y}_c - 1$$

Case 2: $i \neq c$

$$\frac{\partial \mathcal{L}}{\partial z_i} = \hat{y}_i$$

Combined (vector form):

$$\boxed{\frac{\partial \mathcal{L}}{\partial z} = \hat{y} - y}$$

This is remarkably simple! The gradient is just the difference between predicted and true probabilities.

8.10 Why Use Logarithms?

Several practical reasons favor log-probabilities:

1. **Numerical stability**: Products of probabilities become tiny; logs convert to sums

$$\log(p_1 \cdot p_2 \cdots p_n) = \log p_1 + \log p_2 + \cdots + \log p_n$$

2. **Underflow prevention**: $10^{-300} \cdot 10^{-300}$ underflows to 0, but $-300 + (-300) = -600$ is fine
3. **Convexity**: Log-likelihood often has nicer optimization landscapes
4. **Interpretability**: Log-probabilities have units of "bits" or "nats"

8.11 Summary: Loss Function Selection

Summary

The Big Picture:
- Information theory provides principled foundations for loss functions
- Cross-entropy measures how well our predictions match the true distribution
- Minimizing cross-entropy = minimizing KL divergence = maximum likelihood estimation

Loss Function Selection:

Task	Assumption	Loss Function
Binary Classification	Bernoulli	Binary Cross-Entropy
Multi-class Classification	Categorical	Categorical Cross-Entropy
Regression	Gaussian noise	Mean Squared Error

Key Equations:

$$H(P) = -\sum_x P(x) \log P(x) \quad \text{Entropy} \tag{8.13}$$

$$H(P, Q) = -\sum_x P(x) \log Q(x) \quad \text{Cross-Entropy} \tag{8.14}$$

$$D_{KL}(P \| Q) = H(P, Q) - H(P) \quad \text{KL Divergence} \tag{8.15}$$

$$\mathcal{L}_{BCE} = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})] \quad \text{Binary CE} \tag{8.16}$$

$$\mathcal{L}_{CCE} = -\sum_k y_k \log \hat{y}_k \quad \text{Categorical CE} \tag{8.17}$$

8.12 Practical Considerations

8.12.1 Label Smoothing

Instead of hard one-hot labels, use soft labels:

$$y_{\text{smooth}} = (1 - \epsilon) \cdot y + \frac{\epsilon}{K}$$

This prevents overconfident predictions and improves generalization.

8.12.2 Numerical Stability

When computing $\log(\text{softmax}(z))$, use the numerically stable form:

$$\log \text{softmax}(z)_i = z_i - \log \sum_j e^{z_j}$$

Frameworks like PyTorch provide `log_softmax` and `CrossEntropyLoss` that handle this automatically.

Chapter 9

Practical Deep Learning

Key Concept

Moving from theory to implementation requires understanding **automatic differentiation** (autograd), which is the computational engine behind modern deep learning frameworks. This chapter bridges the gap between mathematical understanding and practical implementation.

9.1 From Theory to Implementation

9.1.1 Three Types of Differentiation

We've derived backpropagation mathematically. Now we need to implement it computationally. There are three approaches:

1. **Symbolic Differentiation**: Manipulate mathematical expressions
   - Use symbols and calculus rules to derive exact derivatives
   - Example: $\frac{d}{dx}(x^2) = 2x$
   - Pro: Exact results
   - Con: Expression explosion for complex functions; inefficient
2. **Numerical Differentiation**: Approximate using finite differences
   - Compute $f'(x) \approx \frac{f(x+\epsilon)-f(x)}{\epsilon}$
   - Pro: Simple to implement
   - Con: Slow (requires many function evaluations); numerically unstable
3. **Automatic Differentiation (Autograd)**: Best of both worlds
   - Computes exact derivatives efficiently
   - Breaks computation into elementary operations
   - Applies chain rule automatically
   - This is what PyTorch, TensorFlow, JAX use!

9.1.2 Numerical Differentiation Details

Forward Difference:

$$f'(x) \approx \frac{f(x+\epsilon) - f(x)}{\epsilon}$$

Central Difference (more accurate):

$$f'(x) \approx \frac{f(x+\epsilon) - f(x-\epsilon)}{2\epsilon}$$

```python
def numerical_gradient(f, x, epsilon=1e-5):
    """Compute gradient numerically using central difference."""
    grad = np.zeros_like(x)
    for i in range(len(x)):
        x_plus = x.copy()
        x_minus = x.copy()
        x_plus[i] += epsilon
        x_minus[i] -= epsilon
        grad[i] = (f(x_plus) - f(x_minus)) / (2 * epsilon)
    return grad
```

Listing 9.1: Numerical Differentiation

Problem: For n parameters, this requires $2n$ function evaluations. With millions of parameters, this is far too slow!

9.2 Automatic Differentiation (Autograd)

9.2.1 The Core Idea

Autograd works by:

1. **Recording** operations in a computational graph during the forward pass
2. **Traversing** the graph backward to compute gradients
3. Each operation stores its **local gradient** and how to compute it

9.2.2 Forward Mode vs. Reverse Mode

- **Forward Mode**: Computes derivatives of all outputs w.r.t. *one* input
  - Good when: many outputs, few inputs
  - Complexity: $O(\text{inputs})$ passes
- **Reverse Mode**: Computes derivatives of *one* output w.r.t. all inputs
  - Good when: one output (loss), many inputs (parameters)
  - Complexity: $O(1)$ pass for all gradients
  - **This is what backpropagation uses!**

> Reverse-mode automatic differentiation is **exactly** backpropagation. When you call `loss.backward()` in PyTorch, you're running reverse-mode autograd.

9.3 Building a Simple Autograd System

Let's build a minimal autograd engine to understand how frameworks like PyTorch work.

9.3.1 The Value Class

```python
class Value:
    """A scalar value that tracks gradients."""

    def __init__(self, data, _children=(), _op=''):
        self.data = data
        self.grad = 0.0  # Gradient w.r.t. this value

        # For building the computational graph
        self._backward = lambda: None  # How to compute parent gradients
        self._prev = set(_children)    # Parent nodes
        self._op = _op                 # Operation that created this

    def __add__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data + other.data, (self, other), '+')

        def _backward():
            # Gradient of a+b w.r.t. a is 1, w.r.t. b is 1
            self.grad += 1.0 * out.grad
            other.grad += 1.0 * out.grad
        out._backward = _backward

        return out

    def __mul__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data * other.data, (self, other), '*')

        def _backward():
            # Gradient of a*b w.r.t. a is b, w.r.t. b is a
            self.grad += other.data * out.grad
            other.grad += self.data * out.grad
        out._backward = _backward

        return out

    def __pow__(self, n):
        out = Value(self.data ** n, (self,), f'**{n}')

        def _backward():
            # Gradient of x^n w.r.t. x is n*x^(n-1)
            self.grad += n * (self.data ** (n - 1)) * out.grad
        out._backward = _backward

        return out

    def tanh(self):
        x = self.data
        t = (math.exp(2*x) - 1) / (math.exp(2*x) + 1)
        out = Value(t, (self,), 'tanh')

        def _backward():
            # Gradient of tanh(x) is 1 - tanh(x)^2
            self.grad += (1 - t**2) * out.grad
        out._backward = _backward
```

```python

        return out

    def backward(self):
        """Run backpropagation from this node."""
        # Topological sort
        topo = []
        visited = set()
        def build_topo(v):
            if v not in visited:
                visited.add(v)
                for child in v._prev:
                    build_topo(child)
                topo.append(v)
        build_topo(self)

        # Backprop
        self.grad = 1.0  # dL/dL = 1
        for node in reversed(topo):
            node._backward()

    def __repr__(self):
        return f"Value(data={self.data:.4f}, grad={self.grad:.4f})"

    # Additional operations
    def __radd__(self, other): return self + other
    def __rmul__(self, other): return self * other
    def __neg__(self): return self * -1
    def __sub__(self, other): return self + (-other)
```

Listing 9.2: Simple Autograd Engine: Value Class

9.3.2 Using Our Autograd Engine

```python
# Define values
a = Value(2.0)
b = Value(-3.0)
c = Value(10.0)

# Build expression: d = a*b + c
d = a * b + c

# Backward pass
d.backward()

# Check gradients
print(f"a.grad = {a.grad}")  # dd/da = b = -3
print(f"b.grad = {b.grad}")  # dd/db = a = 2
print(f"c.grad = {c.grad}")  # dd/dc = 1
```

Listing 9.3: Example: Computing Gradients

Output:

```
a.grad = -3.0
b.grad = 2.0
```

```
c.grad = 1.0
```

9.4 Building a Neural Network from Scratch

9.4.1 Neuron Class

```python
import random

class Neuron:
    """A single neuron with weights, bias, and activation."""

    def __init__(self, nin, activation='tanh'):
        # Initialize weights randomly
        self.w = [Value(random.uniform(-1, 1)) for _ in range(nin)]
        self.b = Value(0.0)
        self.activation = activation

    def __call__(self, x):
        # Weighted sum: w.x + b
        act = sum((wi * xi for wi, xi in zip(self.w, x)), self.b)

        # Apply activation
        if self.activation == 'tanh':
            return act.tanh()
        elif self.activation == 'linear':
            return act
        else:
            raise ValueError(f"Unknown activation: {self.activation}")

    def parameters(self):
        return self.w + [self.b]
```

Listing 9.4: Neuron Implementation

9.4.2 Layer Class

```python
class Layer:
    """A layer of neurons."""

    def __init__(self, nin, nout, activation='tanh'):
        self.neurons = [Neuron(nin, activation) for _ in range(nout)]

    def __call__(self, x):
        outputs = [n(x) for n in self.neurons]
        return outputs[0] if len(outputs) == 1 else outputs

    def parameters(self):
        return [p for n in self.neurons for p in n.parameters()]
```

Listing 9.5: Layer Implementation

9.4.3 MLP Class

```python
class MLP:
    """Multi-Layer Perceptron."""

    def __init__(self, nin, layer_sizes):
        sizes = [nin] + layer_sizes
        self.layers = []

        for i in range(len(layer_sizes)):
            # Use 'linear' for output layer, 'tanh' for hidden
            activation = 'linear' if i == len(layer_sizes) - 1 else 'tanh'
            self.layers.append(Layer(sizes[i], sizes[i+1], activation))

    def __call__(self, x):
        for layer in self.layers:
            x = layer(x)
        return x

    def parameters(self):
        return [p for layer in self.layers for p in layer.parameters()]

    def zero_grad(self):
        for p in self.parameters():
            p.grad = 0.0
```

Listing 9.6: Multi-Layer Perceptron

9.5 The Training Loop

```python
# Dataset: simple XOR-like problem
X = [
    [2.0, 3.0, -1.0],
    [3.0, -1.0, 0.5],
    [0.5, 1.0, 1.0],
    [1.0, 1.0, -1.0],
]
y = [1.0, -1.0, -1.0, 1.0]  # Target outputs

# Create network: 3 inputs -> 4 hidden -> 4 hidden -> 1 output
model = MLP(3, [4, 4, 1])

# Training hyperparameters
learning_rate = 0.1
epochs = 100

# Training loop
for epoch in range(epochs):
    # Forward pass: compute predictions
    predictions = [model(x) for x in X]

    # Compute loss (MSE)
    loss = sum((pred - target)**2 for pred, target in zip(predictions, y))

```

```python
    # Zero gradients (important!)
    model.zero_grad()

    # Backward pass
    loss.backward()

    # Gradient descent update
    for p in model.parameters():
        p.data -= learning_rate * p.grad

    # Print progress
    if epoch % 10 == 0:
        print(f"Epoch {epoch}, Loss: {loss.data:.4f}")

# Final predictions
print("\nFinal predictions:")
for x, target in zip(X, y):
    pred = model(x)
    print(f"  Input: {x} -> Pred: {pred.data:.3f}, Target: {target}")
```

Listing 9.7: Complete Training Loop

Sample Output:

```
Epoch 0, Loss: 4.5231
Epoch 10, Loss: 1.2847
Epoch 20, Loss: 0.3412
Epoch 50, Loss: 0.0128
Epoch 90, Loss: 0.0012

Final predictions:
  Input: [2.0, 3.0, -1.0] -> Pred: 0.989, Target: 1.0
  Input: [3.0, -1.0, 0.5] -> Pred: -0.992, Target: -1.0
  Input: [0.5, 1.0, 1.0] -> Pred: -0.985, Target: -1.0
  Input: [1.0, 1.0, -1.0] -> Pred: 0.991, Target: 1.0
```

9.6 The Complete Training Algorithm

Neural Network Training

Input: Dataset $\mathcal{D} = \{(x_i, y_i)\}$, architecture, hyperparameters

1. **Initialize** weights and biases randomly
2. **Repeat** for each epoch:
   (a) **Shuffle** the dataset
   (b) **For each mini-batch**:
      i. **Forward pass**: Compute predictions $\hat{y} = f_\theta(x)$
      ii. **Compute loss**: $\mathcal{L} = \text{LossFunction}(\hat{y}, y)$
      iii. **Zero gradients**: $\nabla_\theta = 0$
      iv. **Backward pass**: Compute $\nabla_\theta \mathcal{L}$ via autograd
      v. **Update parameters**: $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$
3. **Return** trained parameters θ

Don't forget to zero gradients! Gradients accumulate by default (useful for certain architectures), so you must explicitly reset them before each backward pass.

9.7 Hyperparameters

9.7.1 Key Hyperparameters

Hyperparameter	Typical Range	Notes
Learning rate η	10^{-4} to 10^{-1}	Most important; start with 0.001
Batch size	32, 64, 128, 256	Larger = more stable gradients
Number of layers	2–10+	Deeper = more representation power
Neurons per layer	32–1024	Task-dependent
Epochs	10–1000+	Until validation loss plateaus

9.7.2 Learning Rate

The learning rate η is arguably the most important hyperparameter:

- **Too high**: Overshooting, unstable training, loss explodes
- **Too low**: Very slow convergence, may get stuck
- **Just right**: Steady decrease in loss

Learning Rate Schedules:

- **Step decay**: Reduce by factor every n epochs
- **Exponential decay**: $\eta_t = \eta_0 \cdot \gamma^t$
- **Cosine annealing**: Smooth decrease following cosine curve
- **Warmup**: Start small, increase gradually, then decay

9.8 Common Challenges

9.8.1 Vanishing Gradients

Problem: In deep networks, gradients become exponentially small as they backpropagate, especially with sigmoid/tanh activations.

Why it happens:

- Sigmoid derivative: $\sigma'(x) = \sigma(x)(1 - \sigma(x)) \leq 0.25$
- After L layers: gradient $\propto 0.25^L \to 0$

Solutions:

- Use ReLU activation: $\text{ReLU}'(x) = 1$ for $x > 0$
- Use residual connections (skip connections)
- Careful weight initialization (Xavier, He)
- Batch normalization

9.8.2 Exploding Gradients

Problem: Gradients become extremely large, causing numerical overflow.

Solutions:

- **Gradient clipping**: Cap gradients at maximum value

$$\nabla \leftarrow \min\left(1, \frac{\text{threshold}}{||\nabla||}\right) \cdot \nabla$$

- Proper weight initialization
- Lower learning rate

9.8.3 Overfitting

Problem: Model memorizes training data but generalizes poorly.

Signs: Training loss $\ll$ validation loss.

Solutions:

- **More data**: Always the best solution
- **Regularization**: L2 (weight decay), L1, Dropout
- **Early stopping**: Stop when validation loss increases
- **Simpler model**: Fewer layers/neurons
- **Data augmentation**: Artificially expand dataset

9.8.4 Underfitting

Problem: Model is too simple to capture patterns.

Signs: Both training and validation loss are high.

Solutions:

- More complex model (more layers/neurons)
- Train longer
- Better features
- Reduce regularization

9.9 Weight Initialization

Proper initialization is crucial for training deep networks.

9.9.1 Xavier/Glorot Initialization

For layers with tanh/sigmoid activation:

$$W \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{in} + n_{out}}}, \sqrt{\frac{6}{n_{in} + n_{out}}}\right)$$

9.9.2 He Initialization

For layers with ReLU activation:

$$W \sim \mathcal{N}\left(0, \sqrt{\frac{2}{n_{in}}}\right)$$

Information

Why Initialization Matters Poor initialization can cause:

- All neurons outputting similar values (dead neurons)
- Gradients vanishing or exploding from the start
- Slow or impossible training

Good initialization keeps activations and gradients in a reasonable range throughout the network.

9.10 Regularization Techniques

9.10.1 L2 Regularization (Weight Decay)

Add penalty for large weights:

$$\mathcal{L}_{\text{reg}} = \mathcal{L} + \frac{\lambda}{2} \sum_i w_i^2$$

This encourages smaller weights, which often leads to better generalization.

9.10.2 Dropout

During training, randomly set neurons to zero with probability p:

- Prevents co-adaptation of neurons
- Acts like training an ensemble of networks
- At test time, scale outputs by $(1 - p)$ or use inverted dropout

9.11 Tips for Success

Summary

Practical Guidelines:

1. **Start simple**: Begin with a small network and gradually increase complexity
2. **Overfit first**: Make sure your model can memorize a small batch before scaling up
3. **Monitor everything**: Track training/validation loss, gradients, activations
4. **Use established architectures**: Don't reinvent the wheel; start with proven designs
5. **Normalize inputs**: Zero mean, unit variance
6. **Learning rate is key**: Spend time tuning it (try powers of 10)
7. **Batch normalize**: Often helps training significantly
8. **Use validation set**: Never tune hyperparameters on test set
9. **Be patient**: Training large models takes time
10. **Reproducibility**: Set random seeds for debugging

9.12 From Scratch to Frameworks

The autograd engine we built demonstrates the core concepts. Real frameworks add:

- **GPU acceleration**: Massive parallelism for matrix operations
- **Optimized kernels**: Highly tuned CUDA/cuDNN implementations
- **Advanced optimizers**: Adam, RMSprop, AdaGrad, etc.
- **Built-in layers**: Conv2D, BatchNorm, Transformer, etc.
- **Data pipelines**: Efficient loading and augmentation
- **Distributed training**: Scale across multiple machines

```python
import torch
import torch.nn as nn

# Define network
model = nn.Sequential(
    nn.Linear(3, 4),
    nn.Tanh(),
    nn.Linear(4, 4),
    nn.Tanh(),
    nn.Linear(4, 1)
)

# Data
X = torch.tensor([[2.0, 3.0, -1.0],
                  [3.0, -1.0, 0.5],
                  [0.5, 1.0, 1.0],
                  [1.0, 1.0, -1.0]])
y = torch.tensor([[1.0], [-1.0], [-1.0], [1.0]])

# Training
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)
criterion = nn.MSELoss()

for epoch in range(100):
    pred = model(X)
    loss = criterion(pred, y)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    if epoch % 10 == 0:
        print(f"Epoch {epoch}, Loss: {loss.item():.4f}")
```

Listing 9.8: Same Network in PyTorch

The structure is identical to our from-scratch implementation, but now running efficiently on GPU with production-ready code!

9.13 Key Takeaways

Summary

Core Concepts:

- **Automatic differentiation** (autograd) efficiently computes gradients by recording operations and applying the chain rule
- **Reverse-mode autograd** is exactly backpropagation
- Modern frameworks (PyTorch, TensorFlow) are sophisticated autograd engines with GPU support

The Training Loop:

1. Forward pass (compute predictions)
2. Compute loss
3. Zero gradients
4. Backward pass (compute gradients)
5. Update parameters
6. Repeat

Key Challenges:

- Vanishing/exploding gradients $\rightarrow$ proper initialization, ReLU, normalization
- Overfitting $\rightarrow$ regularization, more data, simpler models
- Hyperparameter tuning $\rightarrow$ learning rate most critical